



ToDoList

Application d'aide à l'organisation et au suivi des tâches quotidiennes

Projet réalisé dans le cadre de la présentation au
Titre Professionnel Développeur Web et Web Mobile

présenté par

Nadezhda TOUVENIN

Sommaire

Introduction	1
Résumé du projet	2
Tableau — compétences mises en œuvre dans mon projet	3
Développer la partie front-end d’une application web ou web mobile sécurisée	3
<i>Installer et configurer son environnement de travail en fonction du projet web ou web mobile</i>	3
<i>Maquetter des interfaces utilisateur web ou web mobile</i>	3
<i>Réaliser des interfaces utilisateur statiques web ou web mobile</i>	3
<i>Développer la partie dynamique des interfaces utilisateur web ou web mobile</i>	4
Développer la partie back-end d’une application web ou web mobile sécurisée	4
<i>Mettre en place une base de données relationnelle</i>	4
<i>Développer des composants d’accès aux données SQL et NoSQL</i>	4
<i>Développer des composants métier coté serveur</i>	5
<i>Documenter le déploiement d’une application dynamique web ou web mobile</i>	5
Cahier des charges	5
Expression du besoin fonctionnel, les objectifs	5
<i>A . Besoins</i>	5
<i>B. Objectifs</i>	6
Public visé	6
Users stories	7
Arborescence	8
MVP	8
Versions ultérieures	9
Description du parcours utilisateur	9

Wireframes.....	11
Charte graphique et logo.....	12
Exemples de maquettes.....	13
Spécifications techniques.....	14
Technologies.....	14
Possibilités de déploiement.....	16
1 . Le déploiement en local.....	16
2. Le déploiement en cloud.....	16
3. Le déploiement en cloud centralisé.....	17
Création de la base de données.....	19
MCD.....	19
MLD.....	20
Le dictionnaire des données.....	21
Routes front et back.....	22
Back-end.....	22
Front-end.....	23
Réalisations personnelles.....	24
1. Le Système de glisser-déposer.....	24
Back-end.....	25
Front-end.....	26
2. La mise en place du CRUD.....	29
Back-end.....	29
Front-end.....	33
Présentations du jeu d'essai.....	39
Vulnérabilités de sécurité et veille.....	43
1. Veille technologique.....	43
A. Symfony et architecture MVC.....	43
B. Doctrine ORM et gestion de la base de données.....	43

C. Accessibilité et expérience utilisateur.....	44
2. Veille de sécurité.....	44
A. Authentification et sécurisation des mots de passe.....	44
B. Sécurisation des formulaires et des requêtes.....	45
Processus de recherche et traduction.....	46
1. Processus.....	46
2. Ressource en anglais.....	47
3. Traduction effectuée.....	47
Conclusion.....	49
ANNEXE.....	50
A – Wireframes supplémentaires.....	50
B – Maquettes supplémentaires.....	55

Introduction

Quand j'avais dix-huit ans, j'ai choisi une filière d'études qui ne me correspondait pas réellement. C'était un choix fait par obligation, "parce qu'il fallait choisir quelque chose", et non un choix de passion. J'y ai investi du temps, de l'énergie et de l'argent... pour finalement comprendre que ce domaine ne reflétait ni mes intérêts ni mes aspirations. Après cette expérience, je me suis fait une promesse : si je reprenais un jour des études, ce serait uniquement pour un métier qui me passionne vraiment.

En réalité, l'univers du numérique m'attirait depuis longtemps. J'ai toujours été fascinée par la façon dont les programmes fonctionnent de l'intérieur, par la logique invisible qui fait vivre une application ou un site web. Mais pendant longtemps, je n'ai pas eu l'opportunité de suivre une formation ou de me reconverter professionnellement dans ce domaine.

Lorsque les circonstances l'ont enfin permis, je me suis lancée sérieusement dans le développement web. Pour être sûre que ce n'était pas une curiosité passagère, j'ai décidé d'apprendre seule. Pendant deux ans et demi, après mes journées de travail au McDonald's, j'étudiais environ deux heures chaque soir. J'étais souvent épuisée, mais malgré la fatigue, j'avais hâte de rentrer chez moi pour ouvrir mon ordinateur et continuer d'apprendre. Je ne me forçais pas : c'était un réel plaisir. Explorer une nouvelle technologie, comprendre comment quelque chose fonctionne, résoudre un problème... tout cela me motivait profondément.

C'est cette capacité à avancer même dans les moments de grande fatigue qui m'a confirmée que c'était la bonne voie pour moi. Quand on aime suffisamment une activité pour y consacrer son temps libre après une journée éprouvante, sans perdre l'envie ni la curiosité, c'est qu'elle a une véritable place dans notre vie. J'ai compris que je voulais en faire mon métier : un métier dans lequel je ne compterais pas les heures, où je pourrais travailler huit heures par jour avec envie et satisfaction, et non par obligation. C'est pour cette raison que j'ai choisi de devenir développeuse web.

En comparant les écoles, DAWAN s'est imposée grâce à son accueil chaleureux, la disponibilité de ses formateurs et la clarté de son programme. C'était le cadre idéal pour structurer et approfondir tout ce que j'avais commencé à apprendre seule.

Pour mon projet, j'ai choisi de développer une application TodoList parmi plusieurs propositions. Cette idée m'a immédiatement parlé, car elle répondait à un besoin concret dans mon quotidien : aider mon mari à mieux organiser ses tâches et ses priorités. Le fait de créer une solution utile et personnelle a rendu l'expérience encore plus motivante. J'ai également souhaité réaliser ce projet seule afin de comprendre l'ensemble du processus : conception, base de données, front-end, back-end, logique métier, sécurité et déploiement.

Ce projet m'a permis d'acquérir des compétences essentielles, de comprendre en profondeur l'architecture de Symfony et de renforcer ma confiance en moi. Passer d'un apprentissage théorique à la création d'une application entièrement fonctionnelle a été une expérience extrêmement formatrice et valorisante.

Résumé du projet

L'idée de ce projet est née d'un besoin réel dans mon environnement familial : mon mari rencontrait souvent des difficultés à organiser ses tâches, à se souvenir des échéances importantes et à visualiser clairement ses priorités. Je voulais donc créer un outil simple et intuitif qui l'aiderait à mieux structurer son quotidien.

En avançant dans ma réflexion, j'ai constaté que ce besoin dépasse largement le cadre familial. Beaucoup de profils rencontrent les mêmes difficultés :

- des élèves et étudiants qui doivent organiser leurs devoirs, projets et révisions ;
- des travailleurs qui gèrent plusieurs tâches en parallèle ;
- des parents qui coordonnent leurs obligations familiales ;
- des personnes actives qui souhaitent mieux structurer leur journée.

C'est pour cette raison que j'ai choisi de développer une application TodoList : une méthode visuelle, universelle et efficace, adaptée à tous ces profils et permettant de gérer facilement les tâches, les priorités et les échéances. Mon objectif était de créer un outil utile, concret et accessible pour différents types d'utilisateurs.

Voici les principales fonctionnalités proposées par l'application :

- Création, modification et suppression des tâches : chaque utilisateur peut gérer ses tâches librement.
- Organisation visuelle des tâches dans quatre colonnes (À faire, En cours, Terminé, Urgent) pour suivre facilement la progression.
- Interface simple, intuitive et entièrement responsive, adaptée aux mobiles, tablettes et ordinateurs.
- Système d'inscription et de connexion sécurisé, garantissant que chaque utilisateur accède uniquement à ses propres tâches.
- Possibilité de réinitialiser son mot de passe en cas d'oubli, grâce à un système d'e-mail de récupération.
- Possibilité de supprimer définitivement son compte, conformément aux bonnes pratiques RGPD.
- Ajout d'une date limite (deadline) pour mieux organiser les activités.

- Déplacement automatique d'une tâche dans la colonne "Urgent" lorsque la date limite arrive, afin de prévenir l'utilisateur qu'une action devient prioritaire.

Tableau — compétences mises en œuvre dans mon projet

Développer la partie front-end d'une application web ou web mobile sécurisée

Compétences professionnelles	Applications dans le projet
Installer et configurer son environnement de travail en fonction du projet web ou web mobile	Avant de débiter le développement de l'application TodoList, j'ai configuré un environnement de travail professionnel sous Windows 11, en utilisant WSL2 avec une distribution Debian, afin de disposer d'un environnement proche de la production. J'ai procédé à l'installation et à la configuration des outils suivants : • PHP 8.3, • Composer pour la gestion des dépendances, • Symfony CLI pour la création et la gestion du projet. Le projet a été initialisé à l'aide de la commande <code>composer create-project symfony/skeleton --webapp</code> afin de disposer d'une base complète incluant les composants nécessaires au développement d'une application web. Pour la gestion de la base de données, j'ai mis en place Docker Compose, permettant d'exécuter un service PostgreSQL de manière isolée et reproductible. La connexion entre l'application Symfony et la base de données a été configurée via le fichier <code>.env.local</code>.
Maquetter des interfaces utilisateur web ou web mobile	Avant de commencer le développement de mon application TodoList, j'ai réalisé un travail de maquettage afin de bien définir le fonctionnement de l'application. J'ai d'abord identifié les besoins des utilisateurs à l'aide de user stories, ce qui m'a permis de comprendre quelles actions devaient être possibles (ajouter une tâche, modifier son statut, la supprimer, etc.). Ensuite, j'ai réalisé des wireframes, en version desktop et mobile, pour organiser les pages et réfléchir au parcours utilisateur. Enfin, j'ai créé des maquettes visuelles et une charte graphique afin d'avoir une vision claire du rendu final avant de passer au développement. Ce travail m'a permis de partir sur une base claire et structurée pour la suite du projet.
Réaliser des interfaces utilisateur statiques web ou web mobile	L'application TodoList s'adresse principalement à un public âgé de 20 à 45 ans, habitué à utiliser des outils numériques au quotidien. Cependant, l'interface a été pensée pour rester simple et intuitive, quel que soit le niveau de maîtrise informatique de l'utilisateur. Les interfaces statiques ont été réalisées en HTML et stylisées à l'aide de la bibliothèque Bootstrap, afin de garantir une mise en page claire, cohérente et responsive.

	Grâce à Bootstrap, l'application s'adapte correctement aux différents formats d'écran (ordinateur et mobile), tout en conservant une bonne lisibilité et une navigation fluide. L'objectif était de proposer une interface facile à comprendre, rapide à utiliser et adaptée à un usage quotidien.
Développer la partie dynamique des interfaces utilisateur web ou web mobile	La partie dynamique de l'application TodoList permet à l'utilisateur d'interagir avec les tâches de manière simple et intuitive. L'utilisateur peut : • créer une tâche, • modifier une tâche, • supprimer une tâche, • déplacer une tâche d'une colonne à une autre grâce à une fonctionnalité de glisser-déposer (drag and drop). Chaque action effectuée entraîne une mise à jour immédiate de l'interface, afin que l'utilisateur visualise directement le résultat de son action. Des messages de confirmation sont affichés pour informer l'utilisateur du bon déroulement des actions réalisées, ce qui améliore l'expérience utilisateur et la compréhension de l'interface. Cette partie dynamique rend l'application interactive, fluide et adaptée à un usage quotidien.

Développer la partie back-end d'une application web ou web mobile sécurisée

Compétences professionnelles	Applications dans le projet
Mettre en place une base de données relationnelle	Dès le début du projet TodoList, j'ai accordé une attention particulière à la structuration des données, afin de disposer d'une base solide et cohérente pour l'application. J'ai commencé par réfléchir aux données nécessaires au bon fonctionnement de l'application, puis j'ai conçu le schéma relationnel en passant par un MCD, un MLD, puis un MPD. Les principales tables concernent les utilisateurs et les tâches, avec la mise en place de clés primaires, de clés étrangères et d'index afin d'assurer l'intégrité et les performances de la base de données. Pour la gestion de la base de données, j'ai choisi d'utiliser PostgreSQL, exécuté dans un conteneur Docker, et connecté à l'application Symfony via la variable d'environnement DATABASE_URL. La création et l'évolution de la base ont été réalisées à l'aide de Doctrine Migrations, ce qui m'a permis de versionner la structure de la base et de la faire évoluer progressivement au cours du projet.
Développer des composants d'accès aux données SQL et NoSQL	Pour accéder aux données de l'application TodoList, j'ai utilisé Doctrine ORM, relié à une base de données PostgreSQL. L'accès aux données se fait à l'aide de <u>repositories</u>, associés aux différentes entités de l'application, ce qui permet de lire et d'écrire les données de manière structurée. Cette organisation facilite la maintenance du code et l'évolution de l'application.

Développer des composants métier coté serveur	Pour développer les composants métier de l'application TodoList, j'ai mis en place la logique métier côté serveur à l'aide du framework Symfony. Les règles de fonctionnement de l'application sont gérées dans des <u>controllers</u>, qui traitent les actions de l'utilisateur et garantissent la cohérence des données. Ces composants métier permettent notamment de contrôler les actions possibles, de vérifier les conditions nécessaires à leur exécution et d'assurer le bon déroulement des opérations. Cette organisation côté serveur permet de centraliser la logique métier, de sécuriser les actions sensibles et de garantir un comportement fiable et cohérent de l'application.
Documenter le déploiement d'une application dynamique web ou web mobile	Pour déployer mon application TodoList en local, j'ai suivi une procédure simple et structurée. Après avoir cloné le projet depuis GitHub, j'ai installé les dépendances avec Composer, puis j'ai configuré les variables d'environnement dans un fichier .env.local (connexion PostgreSQL, secret de l'application, mailer local). La base de données est exécutée via Docker Compose ; une fois PostgreSQL lancé, j'ai créé la base et appliqué toutes les migrations Doctrine. Enfin, j'ai démarré le serveur interne de Symfony (symfony server:start) pour rendre l'application accessible en local. Cette procédure garantit un déploiement reproductible, propre et cohérent avec les bonnes pratiques Symfony.

Cahier des charges

Expression du besoin fonctionnel, les objectifs

A . BESOINS

L'utilisateur moderne fait face à une accumulation importante d'informations, de tâches et de priorités quotidiennes. Sans outil structuré, les tâches sont souvent dispersées entre différentes notes, applications ou supports, ce qui peut entraîner des oublis, un manque de visibilité et une perte d'efficacité.

L'application répond à ce besoin en proposant un espace unique, clair et organisé permettant de regrouper, structurer et suivre toutes les tâches du quotidien. La visualisation sous forme de colonnes facilite la compréhension immédiate de l'avancement des activités, tandis que la date limite permet à l'utilisateur d'identifier rapidement ce qui devient urgent.

Ce besoin fonctionnel s'inscrit dans une logique d'amélioration de l'organisation personnelle et de réduction du stress lié à la gestion du temps.

B. OBJECTIFS

L'objectif principal de l'application est d'aider l'utilisateur à organiser ses tâches de manière simple, claire et efficace, afin de mieux gérer son temps et ses priorités au quotidien.

De façon générale, le projet vise à :

- améliorer l'organisation personnelle ;
- offrir un espace unique pour centraliser les tâches ;
- fournir une vision claire de ce qui doit être fait et de ce qui est urgent ;
- réduire les oublis et le stress lié à la gestion quotidienne

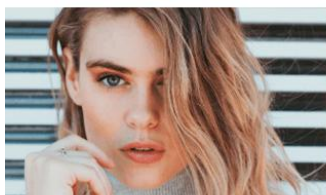
Public visé

Afin d'avoir une bonne représentation de la cible de mes utilisateurs, j'ai établi plusieurs personas.



Nicolas, 32 ans — Toulouse — Employé de bureau

Équipement informatique : ordinateur portable Windows, smartphone Android
Ce qu'il recherche : organiser ses tâches pro/perso rapidement
Sa navigation : connexion → tableau Kanban → déplacement des tâches
Ce qu'il attend : simplicité, rapidité, interface claire
Prérequis : l'application doit être facile à utiliser et réactive sur ordinateur et mobile



Camille, 22 ans — Paris — Étudiante en BTS

Équipement informatique : MacBook, iPhone
Ce qu'elle recherche : gérer ses devoirs et projets avec une vision claire des échéances
Sa navigation : crée plusieurs tâches → consulte « Mes tâches » → vérifie les deadlines
Ce qu'elle attend : interface intuitive + dates limites visibles
Prérequis : la gestion des dates doit être fiable et les pages doivent être lisibles sur petit écran



Julien, 40 ans — Lyon — Père de deux enfants

Équipement informatique : iPad, smartphone Android
Ce qu'il recherche : ne rien oublier entre les rendez-vous, activités et tâches familiales
Sa navigation : ajoute une tâche → définit la date → se repose sur les alertes e-mail si retard
Ce qu'il attend : organisation simple et automatisée
Prérequis : les notifications e-mail doivent être automatiques et fiables

User stories

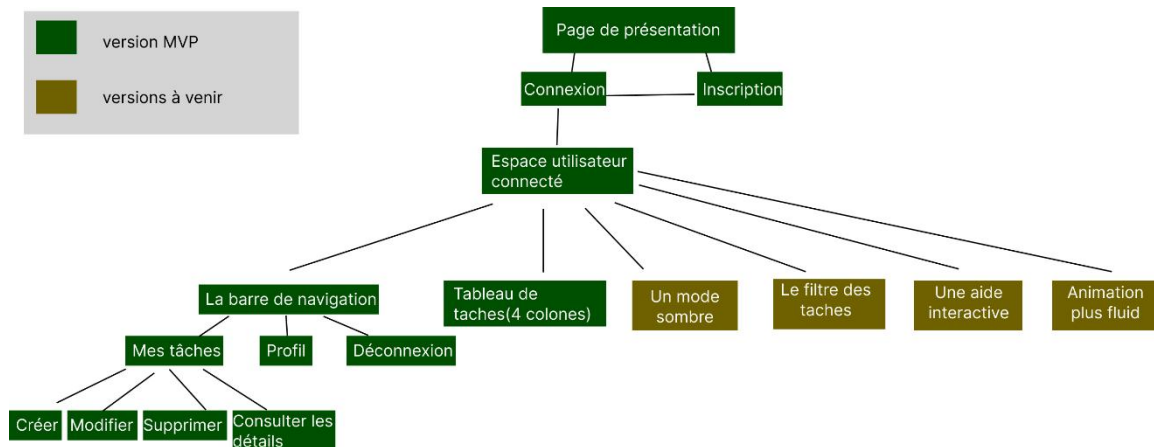
MVP

En tant que...	Je souhaite...	Afin de...
Visiteur	Me créer un compte	Accéder à mon tableau de tâches
Utilisateur connecté	Me connecter à l'application	Retrouver mes tâches et continuer mon organisation
Utilisateur connecté	Créer une tâche	Noter ce que je dois faire
Utilisateur connecté	Modifier une tâche	Corriger ou mettre à jour mes informations
Utilisateur connecté	Supprimer une tâche	Enlever ce qui n'est plus nécessaire
Utilisateur connecté	Déplacer mes tâches dans le tableau (drag & drop)	Organiser visuellement mon travail
Utilisateur connecté	Recevoir un e-mail quand une tâche est en retard	Être informé même si je ne suis pas connecté
Utilisateur connecté	Ajouter un avatar à mon profil	Personnaliser mon espace
Utilisateur connecté	Supprimer mon compte	Contrôler mes données personnelles (RGPD)

Version 1.1-UX/UI

En tant que...	Je souhaite...	Afin de...
Utilisateur connecté	Activer un mode sombre	Améliorer mon confort visuel
Utilisateur connecté	Filtrer ou trier mes tâches	Retrouver rapidement ce dont j'ai besoin
Nouvel utilisateur connecté	Suivre une aide interactive	Comprendre rapidement comment utiliser l'app
Utilisateur connecté	Bénéficier d'animations plus fluides lors du glisser-déposer	Rendre l'organisation de mes tâches plus agréable

Arborescence



MVP

Le MVP correspond à la première version pleinement fonctionnelle de l'application.

Elle contient toutes les fonctionnalités essentielles nécessaires pour permettre à un utilisateur individuel de gérer efficacement ses tâches.

Fonctionnalités incluses dans le MVP :

- Gestion des tâches : création, modification et suppression.
- Tableau visuel à quatre colonnes : À faire, En cours, Terminé, Urgent.
- Glisser-déposer : déplacement intuitif des cartes entre les colonnes.
- Gestion des échéances : date limite + déplacement automatique en « Urgent ».
- Notifications : envoi d'un e-mail en cas de tâche en retard.
- Comptage automatique : mise à jour du nombre de cartes par colonne.
- Compte utilisateur complet : inscription, connexion, réinitialisation du mot de passe.
- Profil : téléversement d'un avatar.
- RGPD : suppression définitive du compte et de toutes les données associées.
- Interface responsive : adaptée aux écrans desktop, tablette et mobile via Bootstrap.

Versions ultérieures

Les versions futures prévoient l'ajout de nouvelles fonctionnalités afin d'enrichir l'expérience, d'améliorer l'ergonomie et d'étendre les possibilités d'utilisation.

Version 1.1 — Améliorations UX/UI

- Mode sombre.
- Animations plus fluides pour le drag & drop.
- Tri et filtrage des tâches (par priorité, date, catégorie).
- Aide interactive pour les nouveaux utilisateurs.

Description du parcours utilisateur

À partir de la page d'accueil, l'utilisateur découvre une présentation générale de l'application ainsi que deux choix : s'inscrire ou se connecter.

S'il sélectionne l'inscription, il est dirigé vers un formulaire où il renseigne ses informations.

Après une inscription validée, il est automatiquement redirigé vers la page de connexion afin d'entrer les identifiants qu'il vient de créer.

Une fois connecté, il accède à la page principale du tableau TodoList, organisée en quatre colonnes (À faire, En cours, Terminé, Urgent).

À cet endroit, il peut visualiser ses tâches sous forme de cartes et les déplacer entre les colonnes grâce au glisser-déposer.

La barre de navigation, accessible en permanence, propose trois sections principales :

1. « Mes tâches »

En cliquant sur ce lien, l'utilisateur est redirigé vers une page listant toutes ses tâches, issues des quatre colonnes, sous forme de lignes.

Sur cette page, il peut :

- voir la liste complète de ses tâches ;
- créer une nouvelle tâche grâce à un bouton situé en haut de la page ;
- modifier une tâche via un bouton dédié ;

- supprimer une tâche ;
- consulter les détails d'une tâche grâce à un lien spécifique.

Cette page permet une gestion plus détaillée et structurée des tâches que la vue Kanban.

2. « Profil »

Dans cette section, l'utilisateur peut téléverser ou changer sa photo de profil (avatar) et supprimer définitivement son compte.

La suppression entraîne la déconnexion immédiate et l'effacement sécurisé de ses données associées.

3. « Déconnexion »

Permet à l'utilisateur de quitter sa session.

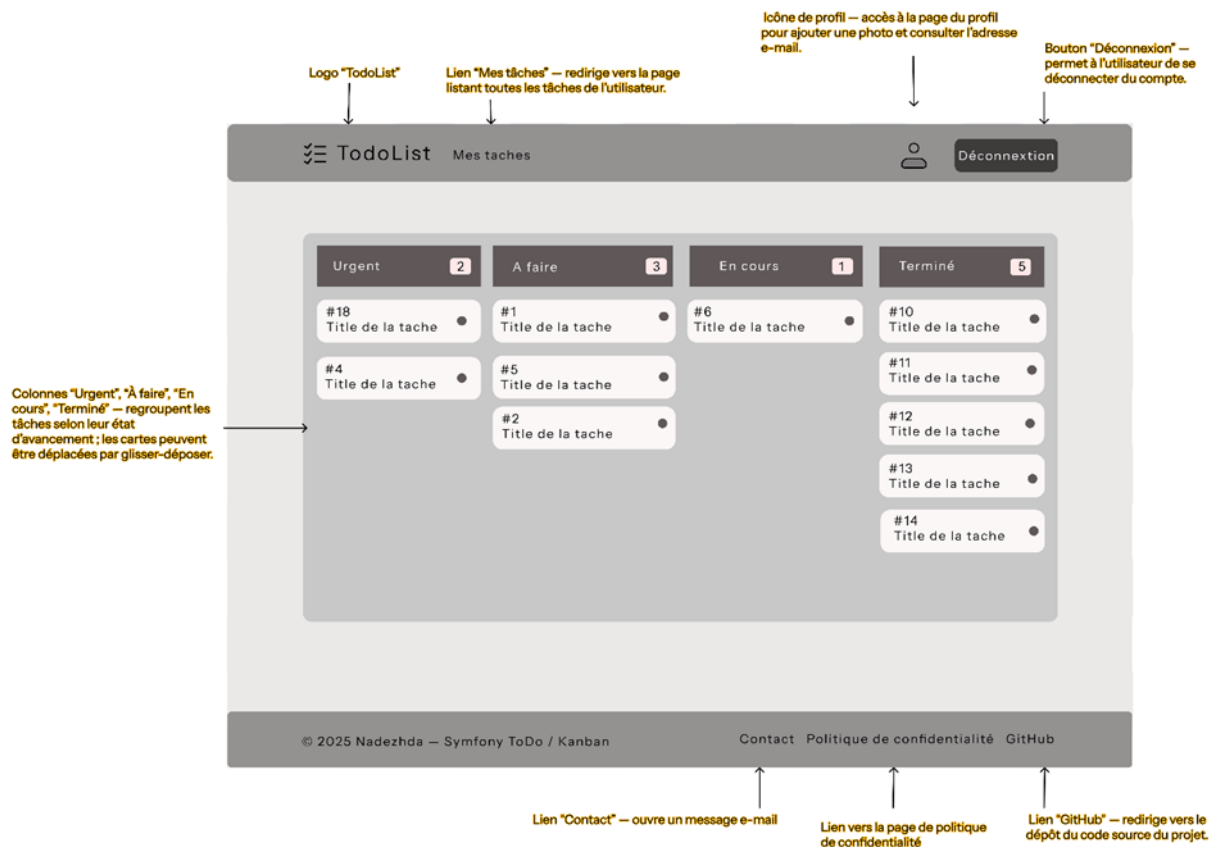
Ce parcours a été conçu pour être intuitif, clair et utilisable facilement sur ordinateur, tablette ou mobile grâce à une interface responsive basée sur Bootstrap.

Wireframes

Pour consulter d'autres wireframes et faciliter la lecture, vous pouvez accéder au lien suivant : <https://www.figma.com/design/W4xf0YtaktgbOlrBwleXnS/TODOLIST?node-id=0-1&p=f&t=ltKa6HRp9BgjCYSW-0>

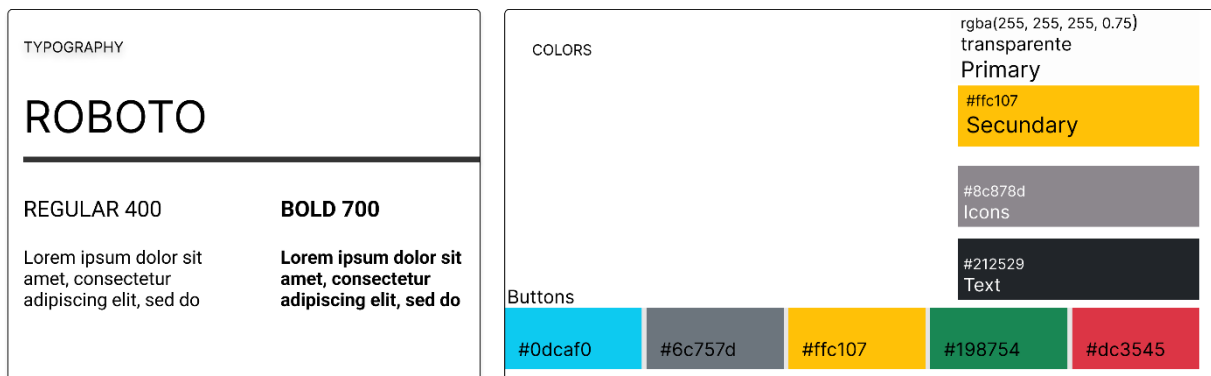
Les autres « Wireframes » peuvent être consultées en Annexe A.

Desktop – la page kanban

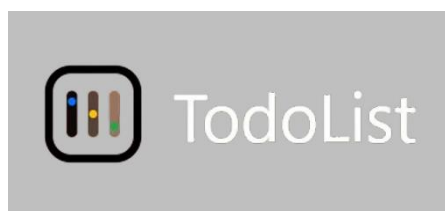


Charte graphique et logo

L'identité visuelle de l'application repose sur une esthétique moderne et épurée, inspirée du glassmorphism afin d'assurer clarté et cohérence sur toutes les pages. La typographie Roboto, utilisée en Regular et Bold, permet une lisibilité optimale pour les titres et contenus. La palette se compose de blancs translucides, d'un jaune accentué et de teintes Bootstrap pour les boutons, garantissant une hiérarchie visuelle cohérente dans l'ensemble de l'interface.

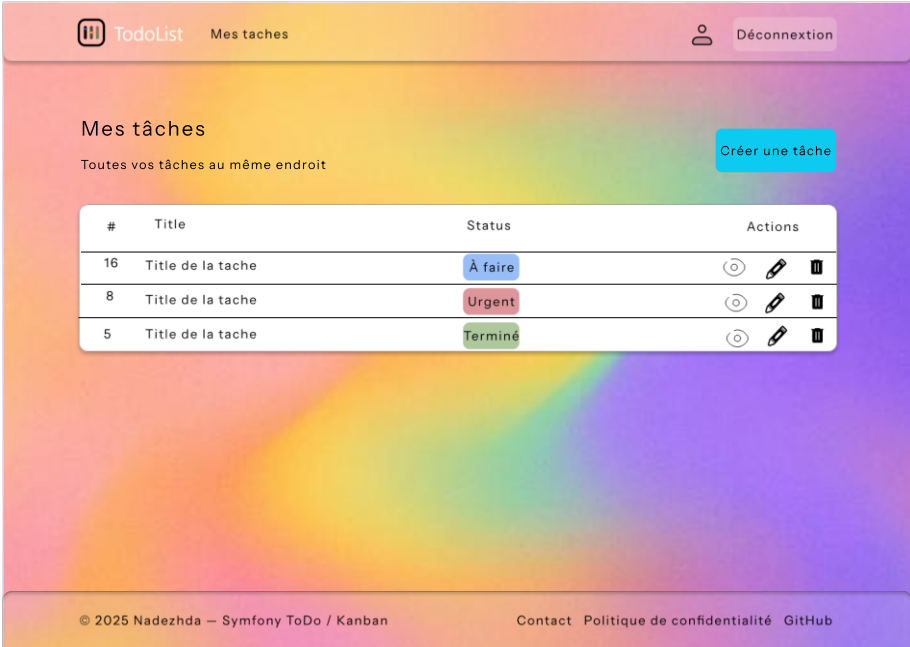


Pour le logo, j'ai choisi une forme carrée représentant l'application, avec trois lignes verticales qui symbolisent les trois colonnes d'un tableau Kanban. Les petits points placés à l'intérieur de ces lignes évoquent les tâches que l'on organise dans chaque colonne. L'ensemble reflète simplement et clairement l'idée de gestion des tâches.



Exemples de maquettes

Les autres « Maquettes » peuvent être consultées en Annexe B.



Spécifications techniques

Technologies :

Front-end

- Twig (moteur de templates de Symfony)
- Bootstrap 5 (mise en page et composants UI)
- Feuille de styles personnalisée en CSS
- JavaScript natif pour le drag & drop des tâches

Back-end

- PHP 8.x avec le framework Symfony
- Doctrine ORM (entités, repositories, requêtes vers la base de données)
- DBAL / PDO pour l'exécution des requêtes SQL générées par Doctrine
- Services Symfony (par ex. service de détection des tâches en retard)
- Symfony Mailer pour l'envoi d'e-mails
- Symfony ResetPasswordBundle pour la gestion de la réinitialisation de mot de passe
- Base de données
- PostgreSQL
- pgAdmin pour l'administration et la visualisation des données
- Migrations Doctrine (création et évolution du schéma via la console Symfony)

Navigateurs compatibles :

Selon les données de 2024, les navigateurs les plus utilisés sont :

Sur desktop :

- Chrome : 64,48 %
- Edge : 12,92 %
- Safari : 9,19 %

- Firefox : 6,44 %
- Opera : 3,39 %

Sur mobile :

- Chrome : 66,41 %
- Safari : 23,44 %
- Samsung Internet : 4,35 %
- Opera : 1,84 %
- UC Browser : 1,55 %

Mon application, développée avec Symfony, Twig, Bootstrap et JavaScript, est compatible avec les versions récentes de ces navigateurs, ce qui permet une utilisation optimale pour la grande majorité des utilisateurs.

Possibilités de déploiement

1 . Le déploiement en local

Avantages	Inconvénients
• Très peu coûteux – pratiquement gratuit, aucun hébergement externe nécessaire. • Installation rapide – l'environnement Symfony + Docker peut être lancé en quelques secondes. • Absence de risques de piratage – l'application n'est pas exposée sur Internet ; elle reste accessible uniquement depuis ma machine ou mon réseau local. • Contrôle total des données – toutes les informations restent stockées en local. • Idéal pour des tests internes – permet de tester l'application en petit groupe, sans infrastructure complexe. • Indépendant d'un hébergeur – aucun service externe n'est requis pour faire fonctionner le projet.	• Accessible uniquement sur le réseau local (= seules les personnes connectées au même réseau Wi-Fi que l'ordinateur peuvent y accéder). • Nécessite une adresse IP locale stable car l'URL peut changer si le routeur réattribue une autre IP (ex. : 192.168.0.10). • URL peu lisible par exemple : <code>http://192.168.0.10:8000</code>. • Mises à jour manuelles je dois gérer moi-même les migrations, services Symfony, mails, redémarrage des conteneurs, etc. • Demande des compétences techniques minimales un utilisateur classique ne pourra pas configurer Docker, PostgreSQL ou Symfony sans aide.

Estimation du coût mensuel : quasiment nul

2. Le déploiement en cloud

Avantages	Inconvénients
• Accessible depuis n'importe où, sans dépendre du réseau local. • URL publique fournie par l'hébergeur (ex : <code>monprojet.render.com</code>). • Possibilité d'utiliser une base PostgreSQL managée, sécurisée et sauvegardée automatiquement. • Peut gérer un nombre d'utilisateurs plus important qu'un déploiement local. • Aucune installation serveur à faire (pas besoin d'installer Linux, PHP, Nginx, PostgreSQL : tout est déjà géré par l'hébergeur). • Compatible avec les besoins de l'application : Symfony, PHP, PostgreSQL, envoi d'e-mails, stockage d'images. • Déploiement facile via Git, souvent automatique (CI/CD intégrée chez Render/Railway).	• Coût mensuel plus élevé qu'un déploiement local. • Nécessite une personne capable de gérer le déploiement, notamment : variables d'environnement, configuration Symfony, gestion du mailer, stockage externe des images. • Les mises à jour doivent être faites manuellement (push Git, migrations Doctrine...). • Obligation d'utiliser des services externes pour les images (ex : Cloudinary ou stockage compatible S3). • Dépendance à l'hébergeur : si Render/Railway a une panne, l'application est indisponible.

Quantité	Nom	Prix
1	Hébergement Cloud PaaS (Render / Railway / Clever Cloud) – App Symfony	7–12 € / mois
1	Base PostgreSQL managée (Render / Railway / Aiven)	0–10 € / mois
1	Service SMTP pour l'envoi d'e-mails (SendGrid / MailerSend : 12k mails gratuits)	gratuit
1	Stockage d'images (Cloudinary ou stockage compatible S3)	0–5 € / mois

TOTAL ESTIMÉ : 7–25 € / mois

(selon le trafic et la quantité d'images stockées)

3. Le déploiement en cloud centralisé

Avantages	Inconvénients
• Beaucoup plus simple d'utilisation pour les utilisateurs finaux : aucune installation, accès direct via une URL officielle. • Les coûts d'hébergement et de maintenance sont assumés par l'organisation (si c'était une entreprise ou une association). • Accessible partout, sur ordinateur comme sur mobile, pour tous les membres de l'organisation. • Nom de domaine clair et professionnel (ex : todolist.entreprise.fr). • Support technique centralisé, géré par une équipe spécialisée (DevOps / administrateurs cloud). • Facilité à déployer les mises à jour grâce à des outils CI/CD automatisés. • Centralisation et uniformité des données : toutes les équipes accèdent à la même base de données et aux mêmes fonctionnalités. • Sécurité renforcée : certificats SSL, sauvegardes, monitoring, gestion des accès, conformité RGPD... • Scalabilité : peut supporter beaucoup plus d'utilisateurs simultanés qu'un hébergement classique.	• Plus coûteux que les autres types d'hébergement (car géré par une équipe + ressources plus importantes). • Nécessite plusieurs personnes techniques pour gérer : • l'infrastructure cloud, • la sécurité, • les environnements, • les bases de données, • la supervision (monitoring). • Dépend fortement de l'organisation centrale : les utilisateurs ne peuvent rien modifier eux-mêmes. • Pour fonctionner à grande échelle, l'application pourrait avoir besoin d'une réécriture partielle, par exemple : • optimiser les requêtes Doctrine, • mieux gérer les images, • séparer front/back, • utiliser un service de queues pour les e-mails. • Plus de données = base PostgreSQL plus grande et plus coûteuse, surtout avec les images.

Dans un modèle cloud centralisé, l'application est hébergée sur une plateforme managée où toute l'infrastructure (serveurs, base de données, sécurité, sauvegardes) est administrée par le fournisseur.

Ce type d'hébergement est généralement plus coûteux qu'un cloud standard, car il inclut :

- le support technique dédié,
- la supervision continue,
- les mises à jour automatiques,
- la haute disponibilité de la plateforme,
- la gestion sécurisée des données utilisateurs.

Selon le fournisseur retenu, le coût mensuel peut varier d'un niveau faible à un niveau assez élevé, en fonction :

- du volume de données,
- du trafic,
- des services managés supplémentaires (backup, scaling automatique, monitoring).

Ce type de solution convient surtout aux organisations multi-sites qui ont besoin d'un accès centralisé et d'un niveau de sécurité élevé.

★ Conclusion — Choix du déploiement retenu

Pour mon projet TodoList, j'ai choisi le déploiement en Cloud classique (option 2) plutôt que la solution cloud centralisé, car :

- il est plus adapté aux besoins réels du projet ;
- il offre un bon équilibre entre coût, simplicité et flexibilité ;
- il ne nécessite pas une équipe d'administration dédiée ;
- il reste suffisant pour un projet individuel ou scolaire.

Le cloud centralisé reste une option intéressante pour de grandes structures, mais il n'est pas nécessaire pour la portée actuelle de mon application.

Création de la base de données

MCD (le Modèle Conceptuel de Données)

Ce modèle me permet de représenter les différentes entités de mon application, les informations qu'elles contiennent et les relations entre elles.

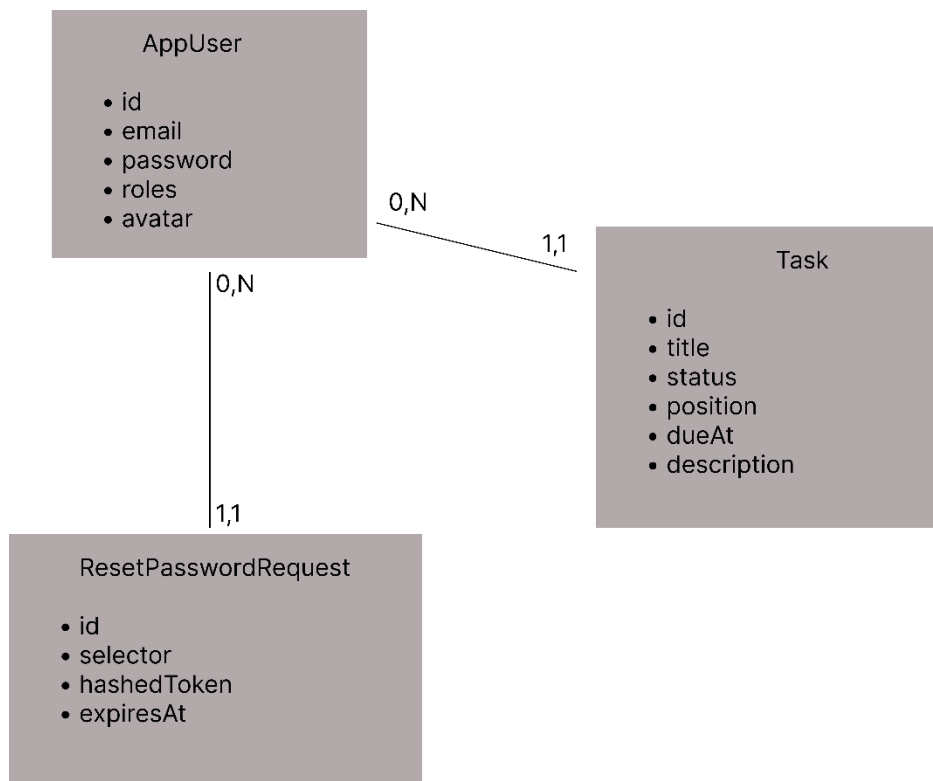
Il s'agit d'une étape indispensable avant la création des tables et de la base de données.

Choix des cardinalités

Les cardinalités ont été définies en fonction des règles de fonctionnement de l'application.

Un utilisateur peut exister dans le système sans avoir encore créé de tâche ou sans avoir effectué de demande de réinitialisation de mot de passe : les cardinalités sont donc (0,N) du côté de l'utilisateur.

En revanche, chaque tâche et chaque demande de réinitialisation de mot de passe sont nécessairement rattachées à un seul utilisateur existant : les cardinalités sont donc (1,1) du côté de ces entités.



MLD

Après avoir défini la représentation visuelle de ma base de données avec le MCD, j'ai créé un MLD (Modèle Logique de Données) pour représenter les relations entre les tables.

APP_USER (id, email, password, roles, avatar)
TASK (id, title, status, position, due_at, description, #owner_id)
RESET_PASSWORD_REQUEST (id, selector, hashed_token, expires_at, #user_id)

Le dictionnaire des données

J'ai ensuite créé un dictionnaire des données pour décrire les champs des tables, leurs types, contraintes et rôles dans la base.

Table app_user

FIELD	TYPE	DETAILS	DESCRIPTION
id	serial	PK, NOT NULL, GENERATED AS IDENTITY	Identifiant unique de l'utilisateur
email	varchar(180)	UNIQUE, NOT NULL	Adresse e-mail de l'utilisateur
roles	json	NOT NULL	Rôles stockés au format JSON
password	varchar(255)	NOT NULL	Mot de passe chiffré
avatar	varchar(255)	DEFAULT NULL	Nom de fichier de l'avatar téléchargé

Table task

FIELD	TYPE	DETAILS	DESCRIPTION
id	serial	PK, NOT NULL, GENERATED AS IDENTITY	Identifiant unique de la tâche
title	varchar(120)	NOT NULL	Titre de la tâche
status	varchar(20)	NOT NULL	Statut de la tâche (todo, doing, done, urgent)
position	int	NOT NULL, DEFAULT 0	Position d'affichage (ordre dans la colonne)
due_at	TIMESTAMP WITHOUT TIME ZONE	DEFAULT NULL	Date d'échéance de la tâche
description	text	DEFAULT NULL	Description optionnelle de la tâche
owner_id	int	NOT NULL, FK → app_user(id)	Référence à l'utilisateur propriétaire

Table reset_password_request

FIELD	TYPE	DETAILS	DESCRIPTION
id	serial	PK, NOT NULL, GENERATED AS IDENTITY	Identifiant unique de la demande
user_id	int	NOT NULL, FK → app_user(id)	Utilisateur ayant demandé la réinitialisation
selector	varchar(20)	NOT NULL	Identifiant public du lien de réinitialisation
hashed_token	varchar(100)	NOT NULL	Jeton privé haché (stocké côté serveur)
requested_at	TIMESTAMP WITHOUT TIME ZONE	NOT NULL	Date et heure de la demande
expires_at	TIMESTAMP WITHOUT TIME ZONE	NOT NULL	Date d'expiration du lien

Routes front et back

1. Back-end

	ENDPOINT	OUTPUT	CONTROLLER	AUTH
GET	/home	Affiche la page d'accueil	HomeController::index	Non-connecté
GET	/login	Affiche le formulaire de connexion	LoginController::login	Non-connecté
POST	/login	Traite le formulaire de connexion, authentifie l'utilisateur	Firewall Symfony (form_login)	Non-connecté
GET	/registration	Affiche le formulaire d'inscription	RegistrationController::register	Non-connecté
POST	/registration	Crée un nouvel utilisateur et envoie l'e-mail de bienvenue	RegistrationController::register	Non-connecté
GET	/reset-password	Affiche le formulaire de demande de réinitialisation de mot de passe	ResetPasswordController::request	Non-connecté
POST	/reset-password	Envoie l'e-mail de réinitialisation de mot de passe	ResetPasswordController::request	Non-connecté
GET	/reset-password/check-email	Affiche la page de confirmation après la demande de réinitialisation	ResetPasswordController::checkEmail	Non-connecté
GET	/reset-password/reset/{token}	Affiche le formulaire pour saisir un nouveau mot de passe	ResetPasswordController::reset	Non-connecté
POST	/reset-password/reset	Enregistre le nouveau mot de passe	ResetPasswordController::reset	Non-connecté
GET	/kanban	Affiche le tableau avec les tâches de l'utilisateur	KanbanController::index	Connecté
POST	/kanban/save-order	Enregistre l'ordre et le statut des tâches (JSON → mise à jour en base)	KanbanController::saveOrder	Connecté
GET	/task	Retourne la liste des tâches de l'utilisateur	TaskController::index	Connecté

GET	/task/new	Affiche le formulaire de création d'une tâche	TaskController::new	Connecté
GET	/task/{id}	Affiche le détail d'une tâche	TaskController::show	Connecté
GET	/task/{id}/edit	Affiche le formulaire d'édition d'une tâche	TaskController::edit	Connecté
POST	/task/{id}/edit	Met à jour une tâche existante	TaskController::edit	Connecté
POST	/task/{id}	Supprime une tâche	TaskController::delete	Connecté
GET	/profile	Affiche la page de profil et le formulaire d'upload d'avatar	ProfileController::index	Connecté
POST	/profile/avatar	Télécharge et enregistre le nouvel avatar de l'utilisateur	ProfileController::upload	Connecté
POST	/profile/delete_account	Supprime le compte utilisateur et déconnecte l'utilisateur	ProfileController::deleteAccount	Connecté
GET	/privacy	Affiche la page de politique de confidentialité	PrivacyController::index	Connecté
POST	/logout	Déconnecte l'utilisateur (géré automatiquement par le firewall Symfony, aucun contrôleur exécuté)	Route app_logout (interceptée par Security)	Connecté

2. Front-end

ROUTE	PAGE	USER
/login	Page de connexion	Non-connecté
/registration	Page d'inscription	Non-connecté
/reset-password	Demande de réinitialisation du mot de passe	Non-connecté
/reset-password/check-email	Confirmation e-mail envoyé	Non-connecté
/reset-password/reset/{token}	Nouveau mot de passe	Non-connecté
/home	Page d'accueil	Non-connecté
/kanban	Affiche le tableau avec les tâches de l'utilisateur	Connecté
/task	Liste des tâches	Connecté
/task/new	Formulaire création d'une tâche	Connecté
/task/{id}	Détail d'une tâche	Connecté
/task/{id}/edit	Modification d'une tâche	Connecté
/profile	Page profil + téléversement avatar	Connecté
/profile/avatar	Upload avatar	Connecté
/profile/delete_account	Suppression du compte	Connecté
/privacy	Page politique de confidentialité	Connecté

Réalisations personnelles

Au cours de ce projet, j'ai eu l'occasion de travailler sur plusieurs aspects techniques, aussi bien en front-end qu'en back-end. J'ai manipulé le DOM en JavaScript, utilisé Symfony pour la gestion des données et mis en place différentes interactions entre l'interface et le serveur.

Cependant, deux réalisations ont particulièrement retenu mon attention :

- LE SYSTEME DE GLISSER-DEPOSER (drag & drop) permettant de réorganiser les tâches et de synchroniser l'ordre avec le back-end ;
- LA MISE EN PLACE DU CRUD des tâches (création, modification, suppression), essentiel au fonctionnement de l'application.

1. Le Système de glisser-déposer

Dans mon application, chaque utilisateur possède un tableau avec quatre colonnes : Urgent, À faire, En cours, Terminé.

Je voulais permettre à l'utilisateur de réorganiser visuellement ses tâches par simple glisser-déposer (drag & drop), puis enregistrer cet ordre en base de données.

Cette fonctionnalité implique :

- du front-end : manipulation du DOM en JavaScript, gestion des événements de drag & drop, mise à jour du compteur de cartes par colonne ;
- du back-end : route Symfony qui reçoit les nouveaux ordres, vérification du token CSRF, mise à jour des entités Task avec Doctrine et sauvegarde dans PostgreSQL.

Back-end : enregistrement de l'ordre des tâches

Route Symfony /kanban/save-order

```
15 final class KanbanController extends AbstractController
43 #[Route('/kanban/save-order', name: 'kanban_save_order', methods: ['POST'])]
44 public function saveOrder(
45     Request $req,
46     TaskRepository $repo,
47     EntityManagerInterface $em,
48     CsrfTokenManagerInterface $csrf
49 ): JsonResponse {
50     // 1) On récupère le JSON envoyé par le front
51     $data = json_decode($req->getContent(), true) ?? [];
52     $token = (string)($data['_token'] ?? '');
53
54     // 2) Sécurité : vérification du token CSRF
55     if (!$csrf->isTokenValid(new CsrfToken('kanban_order', $token))) {
56         return new JsonResponse(['ok' => false, 'error' => 'bad_csrf'], 400);
57     }
58
59     // 3) Mise à jour des colonnes et des positions
60     $columns = ['todo', 'doing', 'done', 'urgent'];
61     foreach ($columns as $col) {
62         // $data[$col] contient la liste des ID dans cette colonne
63         $ids = array_map('intval', (array)($data[$col] ?? []));
64         foreach ($ids as $i => $id) {
65             // Vérification de l'existence de la tâche et de son appartenance à l'utilisateur courant
66             if ($task = $repo->findOneBy(['id' => $id, 'owner' => $this->getUser(),]))
67             {
68                 $task->setStatus($col);
69                 $task->setPosition($i);
70             }
71         }
72     }
73     // 4) On enregistre toutes les modifications en base PostgreSQL
74     $em->flush();
75     // 5) On renvoie une petite réponse JSON
76     return new JsonResponse(['ok' => true]);
}
```

- Route et méthode HTTP

La route /kanban/save-order accepte uniquement des requêtes POST.

Elle est appelée par le JavaScript lorsqu'un utilisateur termine de déplacer une carte.

- Récupération des données

Le front envoie un JSON avec :

- un champ `_token` (token CSRF) ;
- quatre tableaux : `todo`, `doing`, `done`, `urgent`, contenant chacun la liste des ID des tâches dans l'ordre actuel.
- Sécurité CSRF

J'utilise `CsrfTokenManagerInterface` pour vérifier que le token est valide.

Si le token est mauvais → la route renvoie une erreur 400 et rien n'est sauvegardé.

- Mise à jour avec Doctrine

Pour chaque colonne, je parcours la liste des ID :

- `TaskRepository::find($id)` récupère l'objet `Task` ;
- `setStatus($col)` met à jour la colonne (todo/doing/done/urgent) ;
- `setPosition($i)` stocke la position de la tâche dans cette colonne (0, 1, 2, ...).

À la fin, un seul `flush()` enregistre toutes les modifications en une transaction.

- Réponse JSON

Le contrôleur renvoie simplement `{"ok": true}`.

Le front n'a pas besoin de plus : l'interface est déjà à jour visuellement.

Front-end : drag & drop dans le template Twig

Dans mon template Twig `kanban/index.html.twig`, j'affiche les 4 colonnes et les cartes.

Chaque carte reçoit un attribut `data-id` avec l'ID de la tâche.

J'ajoute aussi un meta pour transmettre le token CSRF au JavaScript :

```
10
11 <meta name="csrf-kanban" content="{{ csrf_token('kanban_order') }}">
12
```

Manipulation du DOM en JavaScript :

```
89 <script>
90 document.addEventListener('DOMContentLoaded', () => { // 1. On attend que le DOM soit construit (les nœuds sont disponibles).
91   const columns = document.querySelectorAll('.kanban-cards'); // 2. On récupère tous les conteneurs-colonnes avec la classe .kanban-cards.
92   let dragged = null; // 3. Variable qui contient la carte actuellement déplacée.
93
94   // correspondance entre colonnes et classes de couleur.
95   const DOT_BY_COL = { todo: 'bg-primary', doing: 'bg-warning', done: 'bg-success', urgent: 'bg-danger' };
96
97   // recolorer le point d'une carte en fonction de sa colonne
98   function applyDotColor(task, colKey) {
99     const dot = task.querySelector('.dot');
100     if (!dot) return;
101     dot.classList.remove('bg-primary', 'bg-warning', 'bg-success', 'bg-danger');
102     dot.classList.add(DOT_BY_COL[colKey] || 'bg-secondary');
103   }
104 }
```

Dans cette partie, je manipule directement le DOM pour récupérer les colonnes, les cartes et pour gérer leurs propriétés visuelles.

La fonction `applyDotColor()` permet de changer dynamiquement la couleur du petit indicateur (« dot ») en fonction de la colonne dans laquelle la carte est déplacée.

Gestion des événements de Drag & Drop :

```
106 // Rendre les cartes déplaçables
107 function bindDraggableCards() {
108     document.querySelectorAll('.kanban-card').forEach(card => {
109         card.setAttribute('draggable', 'true');
110         // ajout un attribut
111
112         card.addEventListener('dragstart', (e) => {
113             dragged = card;
114             card.classList.add('dragging');
115             e.dataTransfer.setData('text/plain', card.dataset.id);
116         });
117
118         card.addEventListener('dragend', () => {
119             dragged?.classList.remove('dragging');
120             dragged = null;
121         });
122     });
123 }
124 bindDraggableCards();
125 // 13. On applique immédiatement les écouteurs à toutes les cartes.
```

Ces événements permettent :

- de mémoriser la carte déplacée (dragstart),
- d'ajouter une classe visuelle (dragging),
- de réinitialiser l'état en fin de déplacement (dragend).

Événements sur les colonnes :

```
// 2.Colonnes qui acceptent le drop
columns.forEach(col => {
    col.addEventListener('dragover', (e) => {
        e.preventDefault();
        if (!dragged) return;
        col.appendChild(dragged);
    });
});
// 14) Pour chaque colonne...
// 15) Événement quand on survole la colonne avec une carte.
// 16) OBLIGATOIRE : autoriser le drop, sinon l'événement drop ne se déclenche pas.
// 17) Si aucune carte n'est déplacée, on ne fait rien.
// 18) On place la carte à la fin de cette colonne.
```

L'événement dragover est indispensable, car sans lui le navigateur n'autorise pas le dépôt d'un élément dans la colonne.

En appelant event.preventDefault(), j'indique explicitement au navigateur que cette zone (la colonne) accepte le drop.

C'est cette instruction qui rend possible l'exécution de l'événement drop par la suite.

Drop : mise à jour couleurs + compteurs + sauvegarde :

```
147 col.addEventListener('drop', () => {
148     // узнаём ключ колонки из её id
149     const colKey = (col.id || '').replace('col-', ''); // 'todo' | 'doing' | 'done' | 'urgent' On récupère la clé de la colonne à partir de son id.
150     if (dragged) applyDotColor(dragged, colKey); // On recolor le point de la carte déposée.
151     updateCounters(); // nous recalculons le nombre de tâches dans la colonne
152
153     saveOrder(); // On sauvegarde l'ordre.
154 });
```

Lors du dépôt (drop) :

- je déduis la colonne (todo, doing, done, urgent),
- je mets à jour la couleur du dot,

- je recalcule les compteurs,
- je sauvegarde le nouvel ordre en base via saveOrder().

Mise à jour des compteurs de cartes :

```

134 // A) Fonction de recalcul des compteurs.
135 function updateCounters() {
136   ['todo', 'doing', 'done', 'urgent'].forEach(k => {
137     const box = document.getElementById('col-' + k);
138     const badge = document.getElementById('count-' + k);
139     if (!box || !badge) return;
140     badge.textContent = box.querySelectorAll('.kanban-card').length;
141   });
142 }
143 // B) Premier calcul des compteurs au chargement.
144 updateCounters();

```

Cette fonction :

- compte le nombre de cartes dans chaque colonne,
- met à jour automatiquement les badges affichés dans l'interface,
- est appelée au chargement et après chaque drop.

Envoi de l'ordre des tâches au back-end Symfony :

```

156 // 3. Sauvegarde de l'ordre
157 function saveOrder() {
158   const csrf = document.querySelector('meta[name="csrf-kanban"]')?.content || '';
159   // 21) On envoie au serveur le nouvel ordre des cartes
160   // 22) On lit le jeton CSRF dans la balise <meta>.
161   // 23) On construit l'objet de données.
162   // 24) On collecte la liste des IDs de la colonne todo.
163   // 25) Pareil pour la colonne doing.
164   // 26) Pareil pour done
165   // 27) Pareil pour urgent.
166   // 28) On envoie une requête POST vers la route de sauvegarde.
167   // 29) Méthode HTTP : POST.
168   // 30) On indique au serveur qu'on envoie du JSON.
169   // 31) On convertit l'objet data en JSON.
170   // 32) On n'analyse pas la réponse ici (optionnel .then()).
171   const data = {
172     _token: csrf,
173     todo: Array.from(document.querySelectorAll('#col-todo .kanban-card')).map(c => c.dataset.id),
174     doing: Array.from(document.querySelectorAll('#col-doing .kanban-card')).map(c => c.dataset.id),
175     done: Array.from(document.querySelectorAll('#col-done .kanban-card')).map(c => c.dataset.id),
176     urgent: Array.from(document.querySelectorAll('#col-urgent .kanban-card')).map(c => c.dataset.id),
177   };
178   fetch('/kanban/save-order', {
179     method: 'POST',
180     headers: { 'Content-Type': 'application/json' },
181     body: JSON.stringify(data)
182   });
183 }

```

Cette fonction réalise plusieurs actions :

1. Elle lit le token CSRF dans une balise <meta>.
2. Elle reconstruit l'ordre de toutes les tâches à partir du DOM.
3. Elle envoie une requête POST vers Symfony avec un JSON contenant :
 - le token CSRF,
 - les 4 listes d'IDs classées par colonne.

4. Le back-end met à jour :

- le statut de la tâche,
- sa position,
- et enregistre le tout dans PostgreSQL.

2. La mise en place du CRUD

Dans mon application, chaque utilisateur peut gérer ses propres tâches : les créer, les afficher, les modifier et les supprimer.

C'est ce qu'on appelle un système de CRUD.

Cette fonctionnalité utilise :

- Back-end : contrôleur Symfony TaskController, formulaire TaskType, Doctrine et base de données PostgreSQL ;
- Front-end : templates Twig pour la liste, le détail, la création, la modification, et une fenêtre modale pour confirmer la suppression.

Back-end : contrôleur Symfony TaskController

Lire la liste des tâches (Read – liste) :

```
14 #[Route('/task')]
15 final class TaskController extends AbstractController
16 {
17     #[Route(name: 'app_task_index', methods: ['GET'])]
18     public function index(TaskRepository $taskRepository): Response
19     {
20         $tasks = $taskRepository->findBy(['owner' => $this->getUser()], ['id' => 'ASC']);
21         return $this->render('task/index.html.twig', [
22             'tasks' => $tasks
23         ]);
24     }
25 }
```

- Je récupère toutes les tâches appartenant à l'utilisateur connecté grâce au TaskRepository.
- Je trie les tâches par ID.
- J'envoie ce tableau de tâches au template task/index.html.twig pour l'affichage.

Créer une tâche (Create) :

```
25
26 #[Route('/new', name: 'app_task_new', methods: ['GET', 'POST'])]
27 public function new(Request $request, EntityManagerInterface $entityManager): Response
28 {
29     $task = new Task();
30     $form = $this->createForm(TaskType::class, $task);
31     $form->handleRequest($request);
32
33     if ($form->isSubmitted() && $form->isValid()) {
34
35         /** @var \App\Entity\AppUser $user */
36         $user = $this->getUser();
37         $task->setOwner($user);
38
39         $entityManager->persist($task);
40         $entityManager->flush();
41
42         return $this->redirectToRoute('app_task_index', [], Response::HTTP_SEE_OTHER);
43     }
44
45     return $this->render('task/new.html.twig', [
46         'task' => $task,
47         'form' => $form,
48     ]);
49 }
50
```

- Je crée un nouvel objet Task.
- Je génère un formulaire Symfony à partir de TaskType.
- Je laisse Symfony remplir l'objet avec les données envoyées par l'utilisateur (handleRequest).
- Si le formulaire est soumis et valide :
 - j'associe la tâche à l'utilisateur connecté (setOwner(\$user)),
 - j'enregistre la tâche en base (persist + flush),
 - je renvoie l'utilisateur vers la liste.

Afficher le détail d'une tâche (Read – détail) :

```
51
52 #[Route('/{id}', name: 'app_task_show', methods: ['GET'])]
53 public function show(Task $task): Response
54 {
55     if ($task->getOwner() !== $this->getUser()) {
56         throw $this->createAccessDeniedException("Vous n'avez pas accès à cette tâche.");
57     }
58
59     return $this->render('task/show.html.twig', [
60         'task' => $task,
61     ]);
62 }
63
```

- Symfony récupère automatiquement l'objet Task correspondant à l'ID présent dans l'URL, sans que j'aie besoin de faire une recherche moi-même dans le repository.
- Avant d'afficher la tâche, je vérifie qu'elle appartient bien à l'utilisateur connecté. Si ce n'est pas le cas, l'accès est refusé.
- Une fois la vérification faite, je passe la tâche au template task/show.html.twig, qui affiche ses informations (titre, statut, description, etc.).

Modifier une tâche (Update) :

```

63
64 #[Route("/{id}/edit", name: 'app_task_edit', methods: ['GET', 'POST'])]
65 public function edit(Request $request, Task $task, EntityManagerInterface $entityManager): Response
66 {
67     $form = $this->createForm(TaskType::class, $task);
68     $form->handleRequest($request);
69
70     if ($task->getOwner() !== $this->getUser()) {
71         throw $this->createAccessDeniedException("Vous n'avez pas accès à cette tâche.");
72     }
73
74     if ($form->isSubmitted() && $form->isValid()) {
75         $entityManager->flush();
76
77         return $this->redirectToRoute('app_task_index', [], Response::HTTP_SEE_OTHER);
78     }
79
80     return $this->render('task/edit.html.twig', [
81         'task' => $task,
82         'form' => $form,
83     ]);
84 }
85

```

- Symfony injecte directement la tâche à modifier (Task \$task) à partir de l'ID.
- J'utilise le même formulaire TaskType, mais cette fois il est pré-rempli.
- Si le formulaire est valide, un simple flush() suffit pour sauvegarder les changements.

Supprimer une tâche (Delete) :

```

86
87 #[Route("/{id}", name: 'app_task_delete', methods: ['POST'])]
88 public function delete(Request $request, Task $task, EntityManagerInterface $entityManager): Response
89 {
90     if ($task->getOwner() !== $this->getUser()) {
91         throw $this->createAccessDeniedException("Vous n'avez pas accès à cette tâche.");
92     }
93     if ($this->isCsrfTokenValid('delete'.$task->getId(), $request->getPayload()->getString('_token'))) {
94         $entityManager->remove($task);
95         $entityManager->flush();
96     }
97
98     return $this->redirectToRoute('app_task_index', [], Response::HTTP_SEE_OTHER);
99 }

```

- Cette route accepte seulement les requêtes POST, pour éviter une suppression par simple lien.
- La suppression est protégée par une double vérification : contrôle de l'appartenance de la tâche à l'utilisateur connecté et validation du jeton CSRF.
- Ensuite, je renvoie l'utilisateur vers la liste des tâches.

formulaire Symfony TaskType :

```
13 class TaskType extends AbstractType
14 {
15     // Cette classe décrit le formulaire utilisé pour créer ou modifier une tâche.
16     public function buildForm(FormBuilderInterface $builder, array $options): void
17     {
18         // On construit ici les différents champs du formulaire "Task".
19         $builder
20             // Champ "title" : titre de la tâche, simple champ texte.
21             ->add('title')
22             // Champ "dueAt" : date à laquelle l'utilisateur souhaite être alerté.
23             // On utilise un DateType pour avoir un champ de type date (calendrier HTML5).
24             ->add('dueAt', DateType::class, [
25                 // 'single_text' permet d'afficher la date dans un seul champ, avec le calendrier HTML5, au lieu de trois champs séparés (jour / mois / année).
26                 'widget' => 'single_text',
27                 // Le champ n'est pas obligatoire : l'utilisateur peut laisser la date vide.
28                 'required' => false,
29                 // Le texte qui sert d'étiquette pour le champ.
30                 'label' => 'Date à laquelle vous souhaitez être alerté',
31             ])
32             // Champ "status" : permet de choisir le statut de la tâche dans une liste.
33             ->add('status', ChoiceType::class, [
34                 'label' => 'Statut',
35                 // Liste des choix possibles dans le menu déroulant. La clé est le texte affiché à l'utilisateur, la valeur est stockée en base
36                 'choices' => [
37                     'À faire' => 'todo',
38                     'En cours' => 'doing',
39                     'Terminé' => 'done',
40                     'Urgent' => 'urgent',
41                 ],
42                 // Texte affiché quand aucun statut n'est encore sélectionné
43                 'placeholder' => 'Sélectionnez un statut',
44                 // Attributs HTML du champ (ici, on ajoute une classe CSS Bootstrap)
45                 'attr' => ['class' => 'form-select'],
46             ])
47
48             // Champ "description" : texte optionnel pour décrire la tâche
49             ->add('description', TextareaType::class, [
50                 // Le champ peut être laissé vide
51                 'required' => false,
52                 // Le texte qui sert d'étiquette pour le champ.
53                 'label' => 'description',
54                 // Attributs HTML supplémentaires pour le <textarea>
55                 'attr' => [
56                     // Nombre de lignes visibles par défaut
57                     'rows' => 3,
58                     // Texte d'aide affiché à l'intérieur du champ quand il est vide.
59                     'placeholder' => 'Décrivez la tâche.',
60                     // classe CSS pour appliquer le style Bootstrap
61                     'class' => 'form-control',
62                 ],
63             ]);
64
65         // Configuration des options du formulaire.
66         // Ici, on indique que ce formulaire travaille avec l'entité Task.
67         // Symfony remplira automatiquement un objet Task avec les données du formulaire
68         public function configureOptions(OptionsResolver $resolver): void // Le type void indique simplement que la fonction ne renvoie rien.
69         {
70             // Elle configure les options du formulaire, mais elle ne retourne pas de valeur.
71             $resolver->setDefaults([
72                 'data_class' => Task::class,
73             ]);
74         }
75     }
76 }
```

- TaskType décrit les champs du formulaire pour créer ou modifier une tâche.
- L'utilisateur peut saisir :
- un titre (title),
- un statut parmi une liste,
- une description,
- une date d'alerte (dueAt), optionnelle.



Ce formulaire est utilisé à la fois dans new() et dans edit().

Front-end : templates Twig

Bouton « Créer une tâche » + liste des tâches

En haut de la page de liste, j'affiche un titre et un bouton pour créer une nouvelle tâche :

```
16
17     <a class=" btn btn-info"
18       | href="{{ path('app_task_new') }}"
19       | aria-label="Créer une nouvelle tâche">Créer une tâche</a>
20
21 </div>
22
```

- Ce bouton envoie l'utilisateur vers la route `app_task_new`.
- Cette route appelle la méthode `new()` du contrôleur et affiche le formulaire de création.

Ensuite, j'affiche la liste des tâches dans un tableau (version desktop) avec les actions :

```
105 {# Bloc affiché uniquement sur les écrans ≥ sm (tablette / desktop).
106 Sur mobile, une autre mise en page est utilisée. #}
107 <div class="d-none d-sm-block">
108     <div class="table-card">
109         <div class="table-responsive">
110             <table class="table align-middle mb-0 rounded-5 overflow-hidden">
111                 <thead>
112                     <tr>
113                         <th scope="col" style="width:72px">#</th> {# Colonne pour l'identifiant de la tâche #}
114                         <th scope="col">Titre</th> {# Colonne pour le titre de la tâche #}
115                         <th scope="col" style="width:160px">Statut</th> {# Colonne pour le statut (badge de couleur) #}
116                         <th scope="col" class="text-center" style="width:160px;">Actions</th> {# Colonne pour les boutons Voir / Modifier / Supprimer #}
117                     </tr>
118                 </thead>
119                 <tbody>
```

```
119 <tbody>
120 {# Boucle qui parcourt toutes les tâches renvoyées par le contrôleur #}
121 {% for task in tasks %}
122     {# 1) On prépare une "clé" normalisée à partir du statut de la tâche.
123     - task.status ?? '' → si le statut est null, on prend une chaîne vide
124     - lower → on met tout en minuscules
125     - replace → on enlève les espaces et les underscores #}
126     {% set code = (task.status ?? '')|lower|replace({' ':' ','_':'_'}) %}
127     {# 2) On récupère les métadonnées du statut dans la constante STATUS.
128     STATUS est un tableau défini côté PHP ou passé au template :
129     STATUS[code] contient par exemple : { label: 'À faire', badge: 'badge-todo' }.
130     Si la clé n'existe pas, on met null. #}
131     {% set meta = STATUS[code] ?? null %}
132     <tr>
133         <td class="text-muted">#{{ task.id }}</td> {# Affiche l'ID de la tâche avec un style gris #}
134         <td class="text-break">{{ task.title }}</td> {# Affiche le titre ; "text-break" permet de couper le texte long #}
135         <td>
136             {% if meta %}
137                 {# Si on a trouvé une entrée dans STATUS :
138                 - meta.badge contient la classe CSS pour la couleur
139                 - meta.label contient le texte lisible pour l'utilisateur #}
140                 <span class="badge status-badge {{ meta.badge }}">{{ meta.label }}</span>
141             {% else %} {# Si le statut n'est pas reconnu, on affiche une version "par défaut" #}
142                 <span class="badge status-badge badge-other">{{ task.status }}</span> {# Badge visuel du statut actuel de la tâche #}
143             {% endif %}
144         </td>
145         <td class="text-end">
146             <div class="d-inline-flex flex-wrap justify-content-end gap-2">
147                 {# Bouton "Voir" : lien vers la page de détail de la tâche #}
148                 <a class="btn btn-sm btn-action btn-show"
149                   title="Voir"
150                   href="{{ path('app_task_show', {'id': task.id}) }}"
151                   aria-label="Voir la tâche #{{ task.id }}">
152                     <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16"
153                       fill="currentColor" class="bi bi-eye" viewBox="0 0 16 16"
154                       aria-hidden="true" focusable="false">
155                       <path d="M16 8s-3-5.5-8-5.5S8 0 8s3 5.5 8 5.5S16 8 16 8M1.73 8a13 13 0 1 1 1.66-2.043C4.12 4.668 5.88 3.5 8 3.5s3.88 1.168 5.27 2.043" />
156                       <path d="M8 5.5a2.5 2.5 0 1 0 0 5 2.5 2.5 0 0 0 0 5M4.5 8a3.5 3.5 0 1 1 7 0 3.5 3.5 0 0 1 0 7" />
157                     </svg>
```


[illegible]

Pour chaque tâche, j'affiche trois actions :

- Voir → appelle la route `app_task_show` (méthode `show()`).
- Modifier → appelle la route `app_task_edit` (méthode `edit()`).
- Supprimer → ouvre une modale de confirmation (formulaire vers `app_task_delete`).

Modale de suppression (task/_delete_form.html.twig) :

```
2  {# Fenêtre modale Bootstrap utilisée pour confirmer la suppression d'une tâche.
3  | L'ID contient task.id pour avoir une modale unique par tâche. #}
4  <div class="modal fade"
5      id="deleteModal-{{ task.id }}"
6      tabindex="-1"
7      role="dialog"
8      aria-modal="true"
9      aria-labelledby="deleteModalLabel-{{ task.id }}"
10     aria-describedby="deleteModalDesc-{{ task.id }}"
11     aria-hidden="true">
```

```

12 | | {# .modal-dialog : conteneur principal de la boîte de dialogue.
13 | | C'est Bootstrap qui applique le positionnement (centrage, largeur, etc.). #}
14 | <div class="modal-dialog">
15 | | {# .modal-content : bloc qui contient visuellement la modale
16 | | (en-tête, corps, pied de page). #}
17 | | <div class="modal-content">
18 | | |
19 | | | <div class="modal-header">
20 | | | {# Titre de la fenêtre modale.
21 | | | | L'ID est utilisé par aria-labelledby ci-dessus pour l'accessibilité. #}
22 | | | <h5 class="modal-title" id="deleteModalLabel-{{ task.id }}">
23 | | | | Confirmer la suppression
24 | | | </h5>
25 | | | {# Bouton de fermeture (croix) fourni par Bootstrap.
26 | | | | data-bs-dismiss="modal" indique à Bootstrap de fermer la modale. #}
27 | | | <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Fermer"></button>
28 | | | </div>
29 | | | {# ----- Corps de la modale ----- #}
30 | | | <div class="modal-body" id="deleteModalDesc-{{ task.id }}">
31 | | | | Êtes-vous sûr de vouloir supprimer cette tâche ?
32 | | | | {# Texte supplémentaire uniquement pour les lecteurs d'écran #}
33 | | | | <span class="visually-hidden">Cette action est irréversible.</span>
34 | | | </div>
35 | | | {# ----- Pied de page : boutons d'action ----- #}
36 | | | <div class="modal-footer">
37 | | | | {# Bouton "Annuler" qui se contente de fermer la modale #}
38 | | | | <button type="button" class="btn btn-secondary text-black" data-bs-dismiss="modal" autofocus>
39 | | | | | Annuler
40 | | | | </button>
41 | | | | {# Formulaire de suppression envoyé en POST vers la route app_task_delete #}
42 | | | | <form method="post" action="{{ path('app_task_delete', {'id': task.id}) }}">
43 | | | | | {# Champ caché contenant le token CSRF.
44 | | | | | | le contrôleur vérifie ce token avant de le supprimer. #}
45 | | | | | <input type="hidden" name="_token" value="{{ csrf_token('delete' ~ task.id) }}">
46 | | | | | {# Bouton qui déclenche réellement la suppression (submit du formulaire) #}
47 | | | | | <button type="submit" class="btn btn-danger text-black">Supprimer</button>
48 | | | | </form>
49 | | | </div>

```

- Le bouton poubelle n'envoie pas tout de suite la requête ⇒ il ouvre d'abord cette modale.
- La modale affiche un message de confirmation.
- Le formulaire en POST envoie l'ID de la tâche et le token CSRF vers la route `app_task_delete`.
- Le contrôleur `delete()` vérifie le token, puis supprime la tâche.

Formulaire de création (`task/new.html.twig`) :

```

2  {# On réutilise le layout global de l'application (barre de navigation, <head>, etc.) #}
3  {% extends 'base.html.twig' %}
4  {# Titre de la page, affiché dans l'onglet du navigateur #}
5  {% block title %}Créer une tâche{% endblock %}
6
7  {% block body %}
8  {# Conteneur principal de la page, avec marges verticales #}
9  <div class="container py-4" role="main" aria-labelledby="page-title">
10
11     {# J'ouvre le formulaire avant la zone où se trouve le bouton Enregistrer,
12     | pour que ce bouton fasse bien partie du formulaire et puisse le soumettre correctement. #}
13     {{ form_start(form, {'attr': {'role': 'form', 'aria-label': 'Formulaire de création de tâche'}}) }}
14
15
16     <div class="d-flex flex-column flex-md-row align-items-md-center justify-content-md-between mb-3 gap-3">
17         <div>
18             <h1 id="page-title" class="page-title m-0">Créer une tâche</h1>
19             <div class="page-subtitle" id="page-subtitle">Renseignez les informations de votre tâche</div>
20         </div>
21         {# Groupe de boutons d'action : retour + enregistrement. "role=group" et aria-label pour l'accessibilité. #}
22         <div class="d-flex justify-content-between gap-2" role="group" aria-label="Actions de la page">
23             <a href="{{ path('app_task_index') }}" {# Lien de retour vers la liste des tâches #}
24                 class="btn btn-secondary"
25                 role="button"
26                 aria-label="Retourner à la liste des tâches">✕ Retour</a>
27
28             <button type="submit" {# Bouton de soumission du formulaire (création de la tâche) #}
29                 class="btn btn-success"
30                 aria-label="Enregistrer cette tâche">
31                 Enregistrer
32             </button>
33         </div>
34     </div>

```

```

35     {# Carte visuelle qui contient les champs du formulaire.
36     | - table-card / bg-white / shadow-sm / rounded-5 : classes de style
37     | - role="region" + aria-labelledby : pour décrire la zone aux lecteurs d'écran. #}
38     <div class="table-card bg-white shadow-sm mx-auto rounded-5 overflow-hidden"
39         role="region"
40         aria-labelledby="page-subtitle">
41         <div class="p-3">
42
43             <div class="mb-3"> {# Je choisis moi-même le texte du label ("Titre") au lieu d'utiliser celui généré par Symfony. #}
44                 {# on relie le label au champ par son id #}
45                 {{ form_label(form.title, 'Titre', {'label_attr': {'class': 'form-label fw-bold', 'for': form.title.vars.id}}) }}
46                 {# Champ de saisie lui-même (input texte). On ajoute des attributs HTML pour le style et l'accessibilité.
47                 | form.title.vars.id ~ '_help' lien avec le texte d'aide ci-dessous #}
48                 {{ form_widget(form.title, {
49                     'attr': {
50                         'class': 'form-control',
51                         'placeholder': 'Entrez un titre',
52                         'aria-required': 'true',
53                         'aria-describedby': form.title.vars.id ~ '_help'
54                     }
55                 }) }} {# Texte d'aide associé au champ "title". "visually-hidden" : visible seulement pour les lecteurs d'écran. #}
56                 <small id="{{ form.title.vars.id }}_help" class="form-text text-muted visually-hidden">
57                     Indiquez le titre de votre tâche.
58                 </small>
59             </div>
60
61             <div class="mb-3"> {# Champ : Statut de la tâche (todo / doing / done / urgent) #}
62                 {{ form_label(form.status, 'Statut', {'label_attr': {'class': 'form-label fw-bold', 'for': form.status.vars.id}}) }}
63                 {{ form_widget(form.status, {
64                     'attr': {
65                         'class': 'form-select',
66                         'aria-required': 'true',
67                         'aria-describedby': form.status.vars.id ~ '_help'
68                     }
69                 }) }}
70                 <small id="{{ form.status.vars.id }}_help" class="form-text text-muted visually-hidden">
71                     Choisissez le statut actuel de votre tâche.
72                 </small>
73             </div>

```

```

74     {# Champ : Description (texte libre, optionnel) #}
75     <div class="mb-3">
76         {{ form_label(form.description, 'Description', {'label_attr': {'class': 'form-label fw-bold', 'for': form.description.vars.id}}) }}
77         {# Zone de texte pour la description de la tâche #}
78         {{ form_widget(form.description, {
79             'attr': {
80                 'class': 'form-control',
81                 'aria-describedby': form.description.vars.id ~ '_help'
82             }) }}
83         <small id="{{ form.description.vars.id }}_help" class="form-text text-muted visually-hidden">
84             Ajoutez une brève description ou des détails.
85         </small>
86     </div>
87     {# Champ : Date d'alerte (dueAt) #}
88     <div class="mb-3">
89         {{ form_label(form.dueAt, 'Date à laquelle vous souhaitez être alerté',
90             {'label_attr': {'class': 'form-label fw-bold', 'for': form.dueAt.vars.id}}) }}
91         {# Widget du champ "dueAt". Dans TaskType, ce champ est de type DateType avec widget "single_text",
92            ce qui génère un input de type "date" avec calendrier HTML5. #}
93         {{ form_widget(form.dueAt, {
94             'attr': {
95                 'class': 'form-control',
96                 'aria-describedby': form.dueAt.vars.id ~ '_help'
97             }) }} {# Texte d'aide pour préciser le rôle de cette date (échéance / alerte) #}
98         <small id="{{ form.dueAt.vars.id }}_help" class="form-text text-muted visually-hidden">
99             Sélectionnez la date limite ou la date d'alerte.
100         </small>
101     </div>
102 </div>
103 </div> {# Fermeture du formulaire : Symfony génère automatiquement les champs cachés nécessaires (par exemple le token CSRF). #}
104 <{{ form_end(form) }}>
105 </div>

```

- Ce template affiche le formulaire de création de tâche.
- Quand l'utilisateur clique sur Enregistrer, les données partent vers la route `app_task_new` en POST.
- C'est cette requête que la méthode `new()` du contrôleur traite.

Détail d'une tâche (task/show.html.twig) :

```

9     <div class="container py-4" role="main" aria-labelledby="page-title">
10         <div class="d-flex flex-column flex-md-row align-items-start align-items-md-between mb-3 gap-2">
11             {# Grâce aux classes Bootstrap :
12             * flex-column → affichage en colonne sur mobile (vertical)
13             * flex-md-row → affichage en ligne sur desktop/tablette (horizontal)
14             Cela rend la page responsive automatiquement #}
15             <h1 class="page-title m-0" id="page-title">Détail de la tâche</h1>
16
17             <div class="d-flex align-items-center gap-2">
18                 <a class="btn btn-secondary"
19                     href="{{ path('app_task_index') }}"
20                     aria-label="Retourner à la liste des tâches">← Retour</a> {# Bouton "Retour" vers la liste des tâches #}
21                 <a class="btn btn-warning"
22                     href="{{ path('app_task_edit', {id: task.id}) }}" {# Lien vers l'édition : on passe l'ID dans l'URL #}
23                     aria-label="Modifier la tâche"> Modifier</a>
24             </div>
25         </div>

```

```

26 {# d-none d-md-block =
27   |   | * d-none → caché sur mobile
28   |   | * d-md-block → visible à partir de 768px #}
29 <div class="table-card bg-white shadow-sm p-0 rounded-5 overflow-hidden">
30   <div class="d-none d-md-block">
31     <table class="table align-middle mb-0">
32       <tbody>
33         <tr>
34           <th style="width:200px" scope="row">ID</th>
35           <td class="text-muted">{{ task.id }}</td>
36         </tr>
37         <tr>
38           <th scope="row">Titre</th>
39           <td class="text-break">{{ task.title }}</td> {# text-break évite que les titres trop longs débordent du tableau #}
40         </tr>
41         <tr>
42           <th scope="row">Statut</th>
43           <td> {# On récupère la valeur du statut en sécurité (si null → '') #}
44             {% set statusText = task.status ?? '' %}
45             {# Selon le statut enregistré en base, on applique un badge couleur #}
46             {% if st == 'todo' %}
47               {% set badgeClass = 'badge-todo' %}
48             {% elseif st == 'doing' %}
49               {% set badgeClass = 'badge-doing' %}
50             {% elseif st == 'done' %}
51               {% set badgeClass = 'badge-done' %}
52             {% elseif st == 'urgent' %}
53               {% set badgeClass = 'badge-urgent' %}
54             {% else %}
55               {% set badgeClass = 'badge-other' %}
56             {% endif %}
57           </td>
58         </tr>
59         <tr>
60           <th scope="row">Description</th>
61           <td class="text-muted text-break">{{ task.description }}</td>
62         </tr>
63       </tbody>
64     </table>
65   </div>

```

- Cette page affiche uniquement les informations d'une tâche.
- En haut, l'utilisateur retrouve les actions : Retour, Modifier.
- Aucune logique métier complexe ici : c'est une vue de consultation.

Présentations du jeu d'essai

Afin de vérifier le bon fonctionnement des principales fonctionnalités de mon application, j'ai mis en place un jeu d'essai basé sur deux actions essentielles du CRUD:

- la création d'une tâche ;
- la suppression d'une tâche.

Ces deux actions sont réalisées via l'interface utilisateur de l'application Symfony, à l'aide de formulaires et d'actions visuelles.

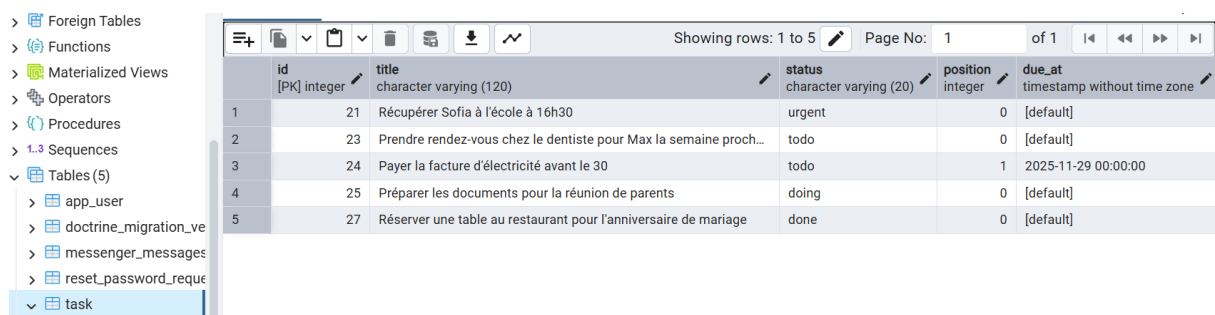
Pour chaque action, je vérifie :

- le comportement de l'application côté interface (avant et après l'action) ;
- la mise à jour effective de la base de données, en observant la table task dans pgAdmin avant et après l'opération.

Ce jeu d'essai permet de valider que les données saisies sont correctement traitées par le contrôleur Symfony, persistées en base de données via Doctrine, puis supprimées lorsque l'action correspondante est effectuée.

Création d'une tâche

Dans un premier temps, j'observe l'état actuel de la table task dans la base de données.



	id [PK] integer	title character varying (120)	status character varying (20)	position integer	due_at timestamp without time zone
1	21	Récupérer Sofia à l'école à 16h30	urgent	0	[default]
2	23	Prendre rendez-vous chez le dentiste pour Max la semaine proch...	todo	0	[default]
3	24	Payer la facture d'électricité avant le 30	todo	1	2025-11-29 00:00:00
4	25	Préparer les documents pour la réunion de parents	doing	0	[default]
5	27	Réserver une table au restaurant pour l'anniversaire de mariage	done	0	[default]

Ensuite, je crée une nouvelle tâche via le formulaire de l'application Symfony.

Modifier la tâche ← Retour ✓ Enregistrer

Titre

Faire les courses pour le dîner (poulet, légumes, pain)

Statut

Urgent

Description

Décrivez la tâche...

Ajoutez ici les détails de votre tâche.

Date à laquelle vous souhaitez être alerté

jj/mm/aaaa

Après validation, la tâche apparaît dans l'interface utilisateur.

Mes tâches Créer une tâche

Toutes vos tâches au même endroit

#	Titre	Statut	Actions
#21	Récupérer Sofia à l'école à 16h30	Urgent	
#23	Prendre rendez-vous chez le dentiste pour Max la semaine prochaine	À faire	
#24	Payer la facture d'électricité avant le 30	À faire	
#25	Préparer les documents pour la réunion de parents	En cours	
#27	Réserver une table au restaurant pour l'anniversaire de mariage	Terminé	
#28	Faire les courses pour le dîner (poulet, légumes, pain)	Urgent	

© 2025 Nadezhda — Symfony ToDo / Kanban Contact Politique de confidentialité GitHub

Enfin, je vérifie dans pgAdmin que la table task a bien été mise à jour :

une nouvelle ligne correspondant à la tâche créée est présente en base de données.

Materialized Views							
Operators							
Procedures							
Sequences							
Tables (5)							
app_user							
doctrine_migration_ve							
messenger_messages							
reset_password_reque							
task							

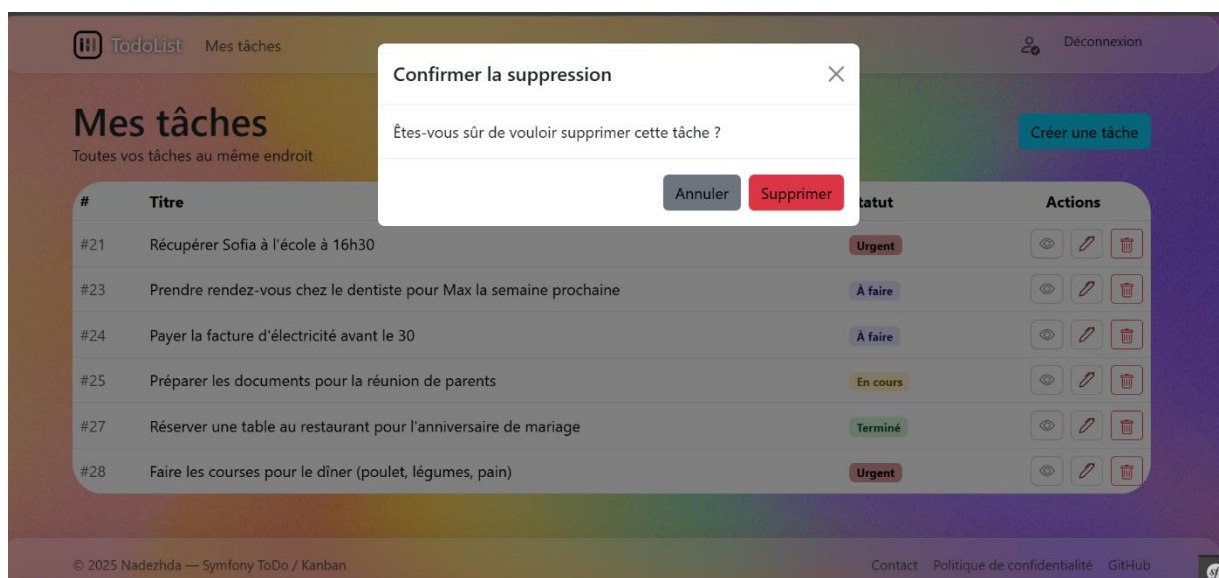
	id [PK] integer	title character varying (120)	status character varying (20)	position integer	due_at timestamp without time zone
1	21	Récupérer Sofia à l'école à 16h30	urgent	0	[default]
2	23	Prendre rendez-vous chez le dentiste pour Max la semaine proch...	todo	0	[default]
3	24	Payer la facture d'électricité avant le 30	todo	1	2025-11-29 00:00:00
4	25	Préparer les documents pour la réunion de parents	doing	0	[default]
5	27	Réserver une table au restaurant pour l'anniversaire de mariage	done	0	[default]
6	28	Faire les courses pour le dîner (poulet, légumes, pain)	urgent	0	[default]

Suppression d'une tâche

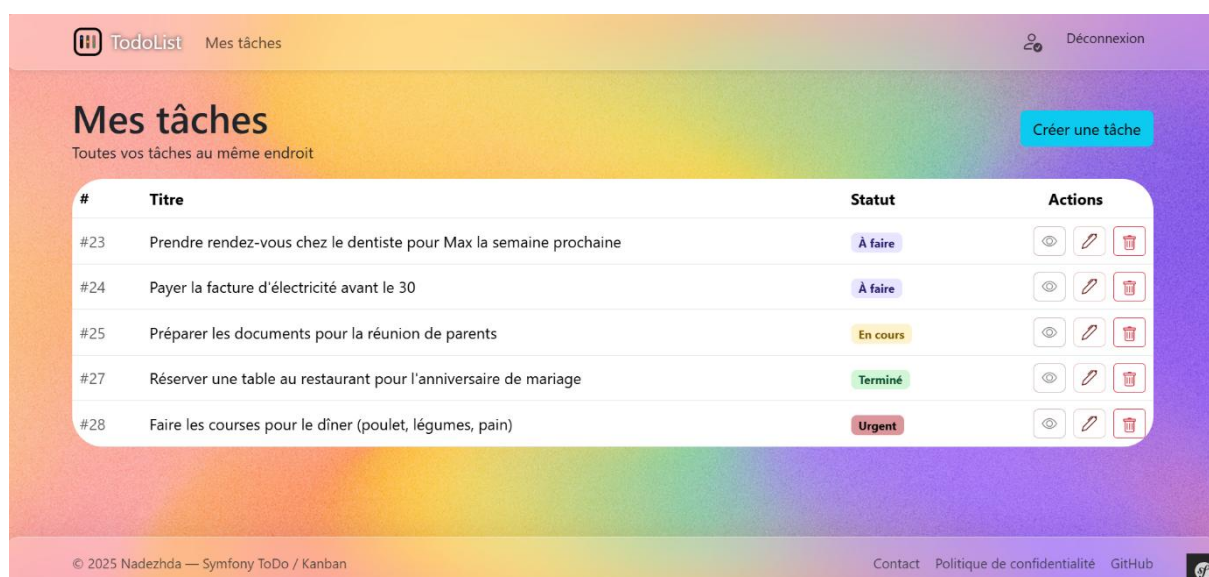
Avant la suppression, la tâche numéro **21** est visible dans l'application et présente dans la table task.



Je lance ensuite la suppression via l'interface, avec une confirmation par modale.



Après validation, la tâche n'apparaît plus dans l'interface utilisateur.



La vérification dans pgAdmin confirme que l'enregistrement a bien été supprimé de la base de données.

	id [PK] integer	title character varying (120)	status character varying (20)	position integer	due_at timestamp without time zone
1	23	Prendre rendez-vous chez le dentiste pour Max la semaine proch...	todo	0	[default]
2	24	Payer la facture d'électricité avant le 30	todo	1	2025-11-29 00:00:00
3	25	Préparer les documents pour la réunion de parents	doing	0	[default]
4	27	Réserver une table au restaurant pour l'anniversaire de mariage	done	0	[default]
5	28	Faire les courses pour le dîner (poulet, légumes, pain)	urgent	0	[default]

1. Veille technologique

A. Symfony et architecture MVC

Pour la réalisation de mon projet TodoList, j'ai choisi d'utiliser le framework Symfony.

Travaillant seule sur ce projet, je me suis appuyée sur les outils et les bonnes pratiques proposés directement par le framework afin de structurer correctement mon application.

Au départ, je me suis surtout posé la question de l'organisation du code et de la manière de séparer les différentes parties de l'application.

L'architecture MVC, proposée nativement par Symfony, m'a permis de distinguer clairement la gestion des données (Model) , la logique applicative (Controller) et l'affichage (View).

Je me suis principalement référée à la documentation officielle de Symfony afin de comprendre comment structurer mes contrôleurs, mes entités et mes vues, sans chercher à mettre en place une architecture trop complexe pour le périmètre du projet.

Sources consultées :

- <https://symfony.com/doc/current/index.html>
- <https://symfony.com/doc/current/controller.html>

B. Doctrine ORM et gestion de la base de données

La gestion des données étant un élément central de l'application (utilisateurs, tâches, statuts), j'ai naturellement utilisé Doctrine ORM, qui est l'outil recommandé avec Symfony.

Je me suis interrogée sur la manière d'interagir avec la base de données, notamment entre l'utilisation de requêtes SQL directes ou d'un ORM.

Doctrine m'a permis de travailler avec des objets PHP tout en laissant le framework gérer la communication avec la base de données.

Cette solution m’a semblé plus adaptée à mon niveau et au temps dont je disposais, tout en restant cohérente avec les bonnes pratiques Symfony.

Elle m’a également permis de limiter les risques d’erreurs et de simplifier la gestion des relations entre les différentes entités.

Pour mettre en place cette partie, je me suis appuyée sur les ressources suivantes :

- <https://symfony.com/doc/current/doctrine.html>
- <https://www.doctrine-project.org/projects/doctrine-orm/en/current/>

C. Accessibilité et expérience utilisateur

Lors du développement de TodoList, j’ai également porté une attention particulière à l’expérience utilisateur et à l’accessibilité de l’interface.

Étant seule sur le projet, j’ai fait le choix d’utiliser Bootstrap, qui permet de créer rapidement une interface responsive et cohérente.

Je me suis interrogée sur la lisibilité des formulaires, la clarté des boutons et la navigation générale de l’application.

Même si toutes les règles d’accessibilité ne peuvent pas être appliquées de manière exhaustive, j’ai essayé de respecter les principes de base, notamment en utilisant des labels clairs, des contrastes suffisants et une structure compréhensible.

Les principales ressources utilisées pour cette réflexion sont :

- <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- <https://developer.mozilla.org/fr/docs/Web/Accessibility>

2. Veille de sécurité

A. Authentification et sécurisation des mots de passe

Une partie importante de ma réflexion a concerné la sécurité des comptes utilisateurs, en particulier l’authentification et le stockage des mots de passe.

Symfony propose un système de sécurité intégré, que j'ai choisi d'utiliser afin de ne pas réimplémenter moi-même des mécanismes sensibles.

Je me suis renseignée sur les bonnes pratiques de hashage des mots de passe et sur la gestion des rôles utilisateurs, afin de limiter les accès aux fonctionnalités sensibles.

Pour cette partie, je me suis appuyée sur les documentations et ressources suivantes :

- <https://symfony.com/doc/current/security.html>
- <https://symfony.com/doc/current/security/passwords.html>

B. Sécurisation des formulaires et des requêtes

Enfin, je me suis intéressée à la sécurisation des formulaires, notamment lors de l'envoi de données sensibles comme les formulaires de connexion ou de création de tâches.

Symfony intègre par défaut des tokens CSRF, que j'ai utilisés afin de protéger l'application contre ce type d'attaques.

J'ai également mis en place une validation des données côté serveur, afin de vérifier les informations envoyées par l'utilisateur.

Lors de mes recherches, j'ai identifié d'autres mécanismes de sécurité, comme le rate-limiting ou des règles CORS plus strictes.

Cependant, compte tenu du périmètre et du temps disponible pour ce projet, ces éléments n'ont pas été implémentés, mais pourraient faire l'objet d'améliorations futures.

Sources consultées :

- <https://symfony.com/doc/current/security/csrf.html>
- <https://owasp.org/www-project-top-ten/>

Processus de recherche et traduction

1. Processus

Dans le cadre de mon projet TodoList, j'utilise principalement les formulaires Symfony, qui intègrent automatiquement la protection CSRF.

Cependant, lors de la mise en place de la fonctionnalité de suppression d'une tâche, j'ai choisi d'utiliser une forme HTML personnalisée, notamment pour l'afficher dans une fenêtre modale Bootstrap.

Dans ce cas précis, la protection CSRF n'est plus gérée automatiquement par Symfony. Il a donc été nécessaire pour moi de comprendre comment intégrer manuellement un token CSRF dans une forme HTML classique et comment le vérifier côté serveur.

Afin de répondre à cette problématique, j'ai commencé par effectuer mes recherches en anglais, car la majorité des ressources techniques et des exemples récents sont disponibles dans cette langue.

The screenshot shows a Bing search results page for the query "symfony csrf form". The search bar at the top shows the query and the number of results (263). Below the search bar, there are tabs for "TOUT", "RECHERCHER", "IMAGES", "VIDÉOS", "CARTES", "ACTUALITÉS", "COPILOT", and "PLUS". The main results section shows a snippet for "CSRF Token in Symfony" with a brief description and a code block for enabling CSRF protection in Symfony. To the right of the main results, there is a sidebar with a list of related searches: "Symfony csrf example", "Symfony form csrf", "Symfony csrf token", "Symfony csrf tutorial", "Symfony csrf protection", "csrf in Symfony", and "csrf web application". Below the main results, there are several search results from various sources, including Symfony Docs, Sling Academy, and DavFi, each with a title, URL, and a brief description.

CSRF Token in Symfony 1 2

A **CSRF token** (Cross-Site Request Forgery token) is a security mechanism used in Symfony to protect web applications from unauthorized actions performed by malicious actors. It ensures that requests to the server originate from trusted sources, preventing attackers from exploiting authenticated sessions.

How CSRF Tokens Work in Symfony

Symfony automatically integrates CSRF protection into its forms. A unique token is generated and embedded as a hidden field in forms. When the form is submitted, Symfony validates the token to confirm the request's authenticity. This prevents unauthorized actions like form submissions from external websites.

Enabling CSRF Protection

CSRF protection is enabled by default in Symfony. You can configure it in the `framework.yaml` file:

```
framework:
  csrf_protection:
    enabled: true
```

Découvrez symfony csrf...

- Symfony csrf example
- Symfony form csrf
- Symfony csrf token
- Symfony csrf tutorial
- Symfony csrf protection
- csrf in Symfony
- csrf web application

Symfony
https://symfony.com › doc › current › security › cs... Traduire ce résultat

How to Implement CSRF Protection (Symfony Docs)
Symfony Forms include CSRF tokens by default and Symfony also checks them automatically for you. So, when using Symfony Forms, you don't have to do anything to be protected against ...

SymfonyConnect
SymfonyConnect is a developer social ...

ESI Fragment
Fortunately, Symfony provides a solution ...

How to Implement CSRF Prot...
Although Symfony Forms provide ...

How to Configure Symfony t...
If your Symfony application runs behind a ...

Symfony1 Legacy
symfony 1.x legacy website The symfony ...

Symfony Blog
Official Symfony Blog: all about Symfony ...

Afficher uniquement les résultats de symfony.com

Sling Academy
https://www.slingacademy.com › article › csrf-in-... Traduire ce résultat

CSRF in Symfony: A Practical Guide (with Examples)
14 janv. 2024 - In this tutorial, we will look at how to handle CSRF protection in Symfony, one of the most popular PHP frameworks. Symfony provides an easy-to-use CSRF token service out of ...

DavFi
https://davfi.fr › formulaire-web-comment-generer-un-jeton-csrf-valide...

Formulaire web : comment générer un jeton CSRF valide en PHP / ...
4 juin 2025 - Comment générer un jeton CSRF sécurisé dans Symfony étape par étape ? Pour

J'ai utilisé des mots-clés simples et ciblés afin d'obtenir des réponses précises sur l'intégration manuelle des tokens CSRF dans Symfony.

J'ai ensuite consulté en priorité la documentation officielle de Symfony, puis complété mes recherches avec d'autres ressources techniques fiables, en tenant compte de la date de publication et de la cohérence des informations.

Cette démarche m'a permis de comprendre le fonctionnement de la protection CSRF en dehors des formulaires Symfony standards et de l'appliquer correctement dans mon application TodoList.

2. Ressource en anglais

Dans le cadre de cette recherche, j'ai consulté la documentation officielle de Symfony concernant la protection CSRF.

L'extrait suivant explique précisément comment générer et vérifier manuellement un jeton CSRF lorsque l'on utilise des formulaires HTML classiques non gérés par le composant Symfony Form :

Although Symfony Forms provide automatic CSRF protection by default, you may need to generate and check CSRF tokens manually for example when using regular HTML forms not managed by the Symfony Form component.

Consider a HTML form created to allow deleting items. First, use the `csrf_token()` Twig function to generate a CSRF token in the template and store it as a hidden form field:

Then, get the value of the CSRF token in the controller action and use the `isCsrfTokenValid()` method to check its validity, passing the same token ID used in the template.

3. Traduction effectuée

Bien que les formulaires Symfony fournissent par défaut une protection CSRF automatique, il peut être nécessaire de générer et de vérifier manuellement des jetons CSRF, par exemple lorsque l'on utilise des formulaires HTML classiques qui ne sont pas gérés par le composant Symfony Form.

Considérons un formulaire HTML permettant de supprimer des éléments. Dans un premier temps, on utilise la fonction Twig `csrf_token()` afin de générer un jeton CSRF dans le template et de le stocker dans un champ de formulaire caché (hidden input).

Ensuite, on récupère la valeur du jeton CSRF dans l'action du contrôleur et on utilise la méthode `isCsrfTokenValid()` pour vérifier sa validité, en utilisant le même identifiant de jeton que celui défini dans le template.

Conclusion

Avant toute chose, je tiens à remercier les formateurs de l'école DAWAN, de véritables professionnels, qui ont su transmettre leurs connaissances avec clarté et bienveillance.

Malgré la barrière de la langue, ils ont toujours pris le temps d'expliquer, de reformuler et de rendre accessibles des notions parfois complexes. Grâce à eux, j'ai pu apprendre des choses que j'aime réellement faire aujourd'hui.

Réaliser ce projet seule a ensuite représenté un véritable défi personnel. Cela n'a pas toujours été simple, mais ce travail m'a permis de développer mon autonomie, ma patience et ma capacité à chercher des solutions par moi-même.

Ce projet m'a appris à ne pas abandonner face aux difficultés, à m'appuyer sur la documentation officielle et à transformer les problèmes rencontrés en solutions concrètes au sein de mon application TodoList.

Enfin, cette expérience m'a permis de gagner en confiance, tant sur le plan technique que personnel, et de confirmer mon envie de continuer à évoluer dans le domaine du développement web.

ANNEXE

Les repos du projet sont accessibles à cette adresse:

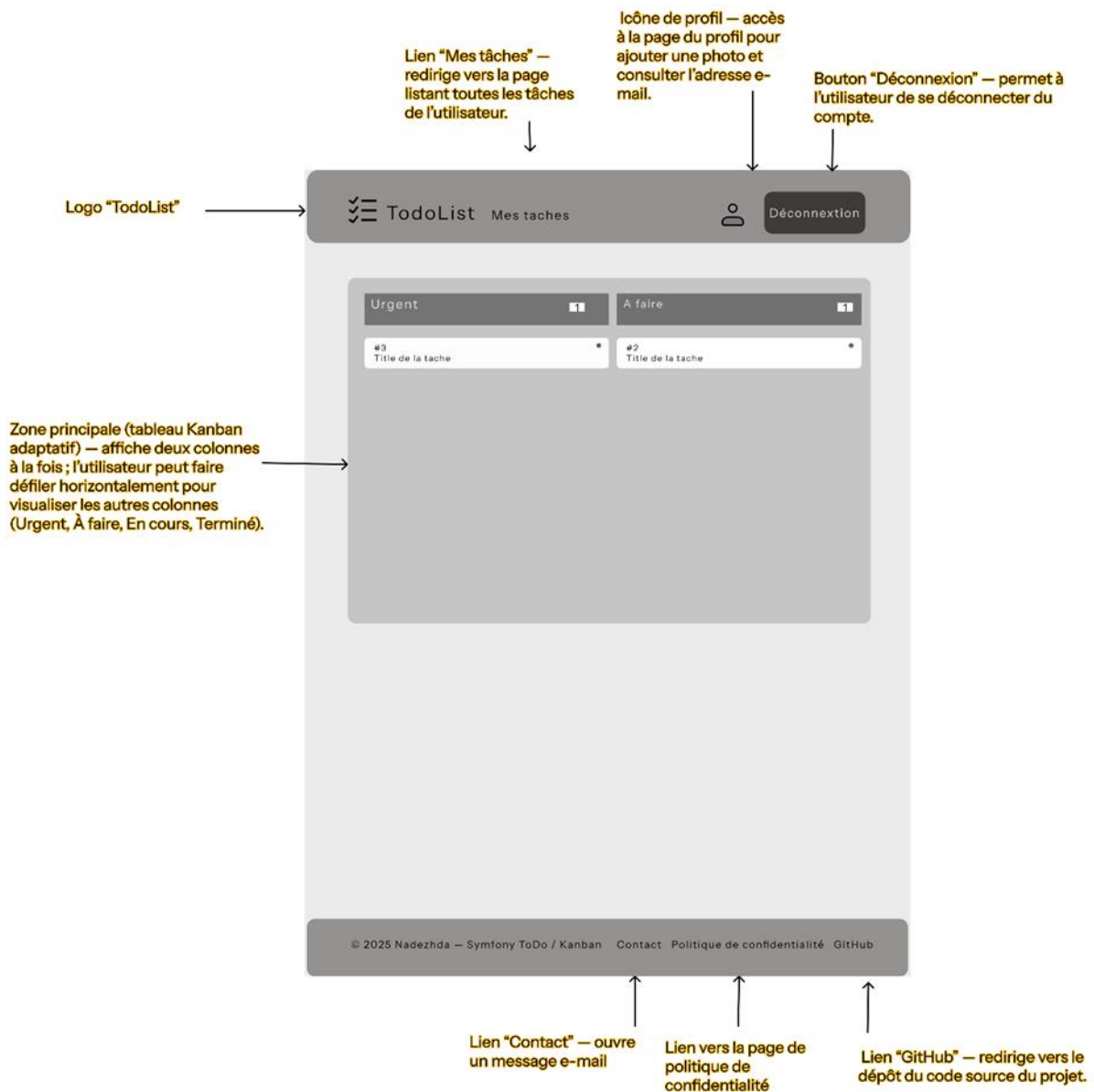
<https://github.com/malygina777/SymfonyToDo.git>

Le site est déployé sur un serveur distant et peut être consulté à l'adresse suivante :

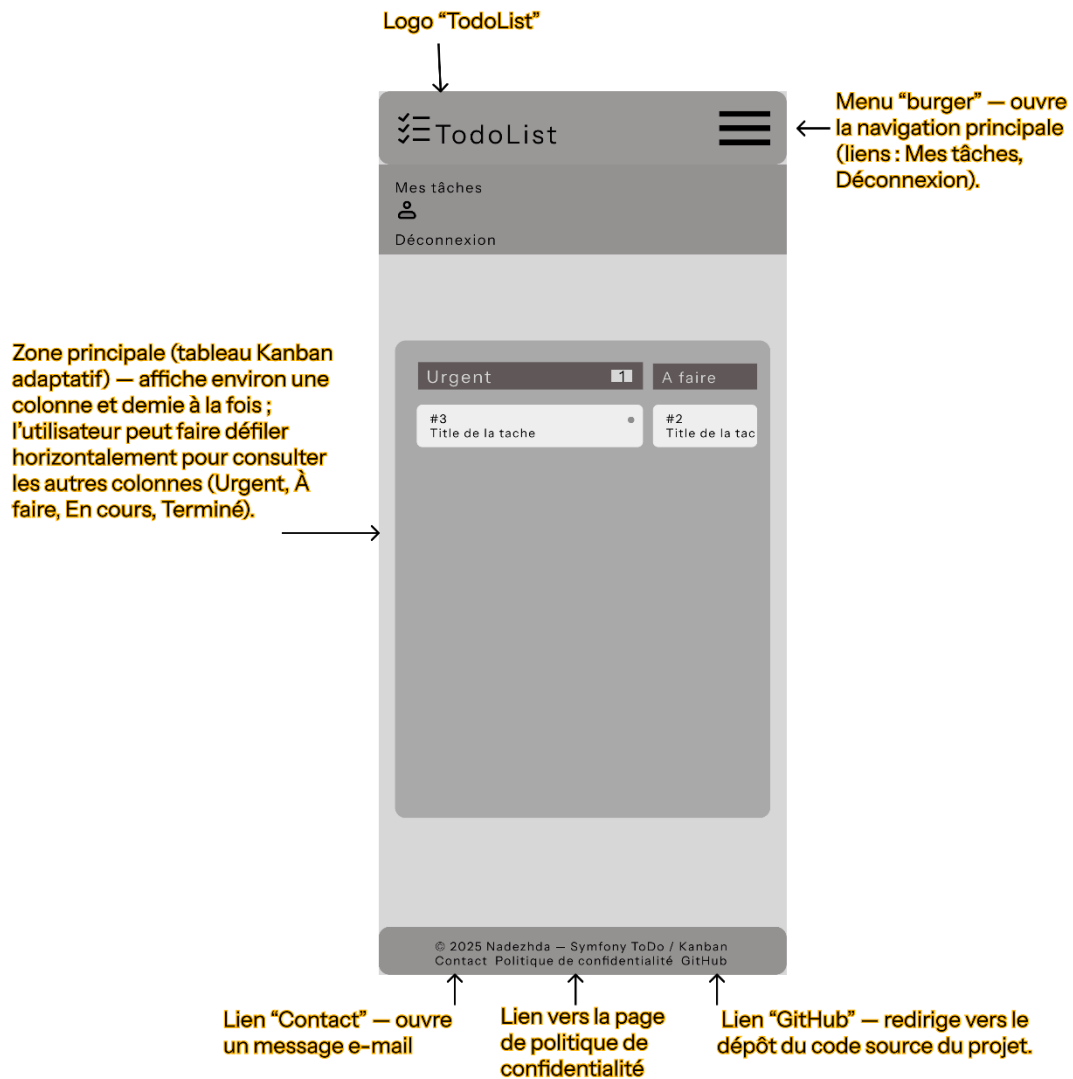
<https://symfonytodolist.alwaysdata.net>

Annexe A – Wireframes supplémentaires

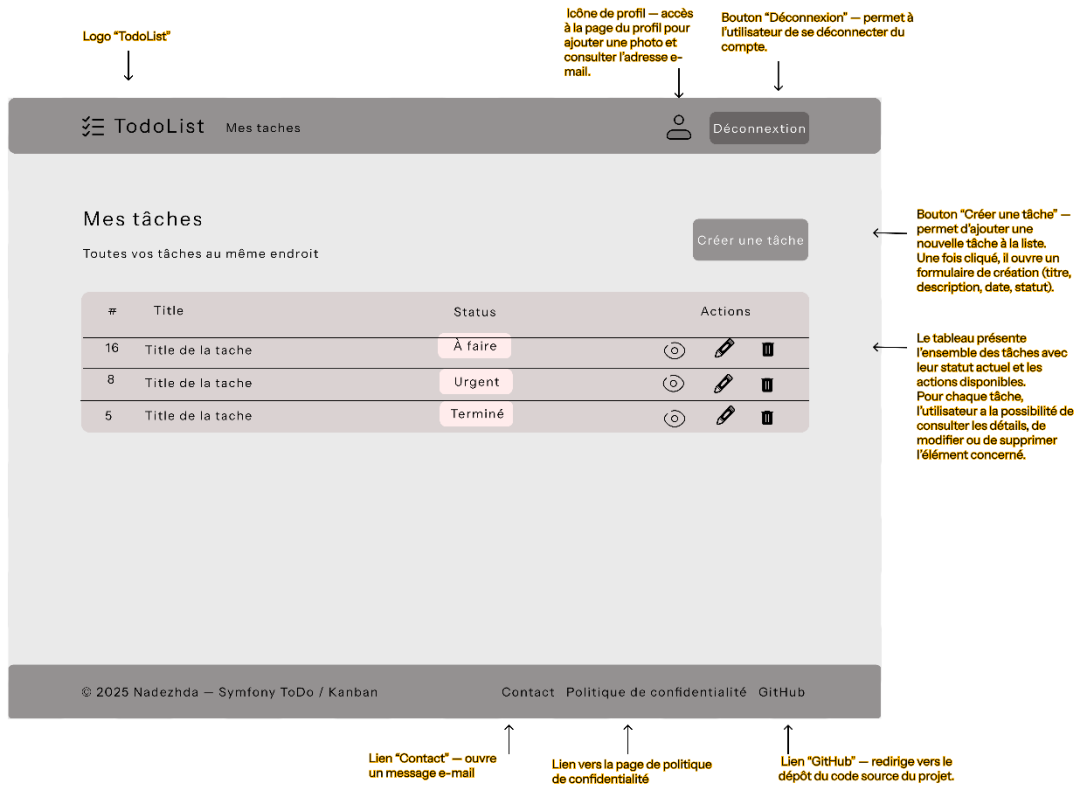
Tablette – la page kanban



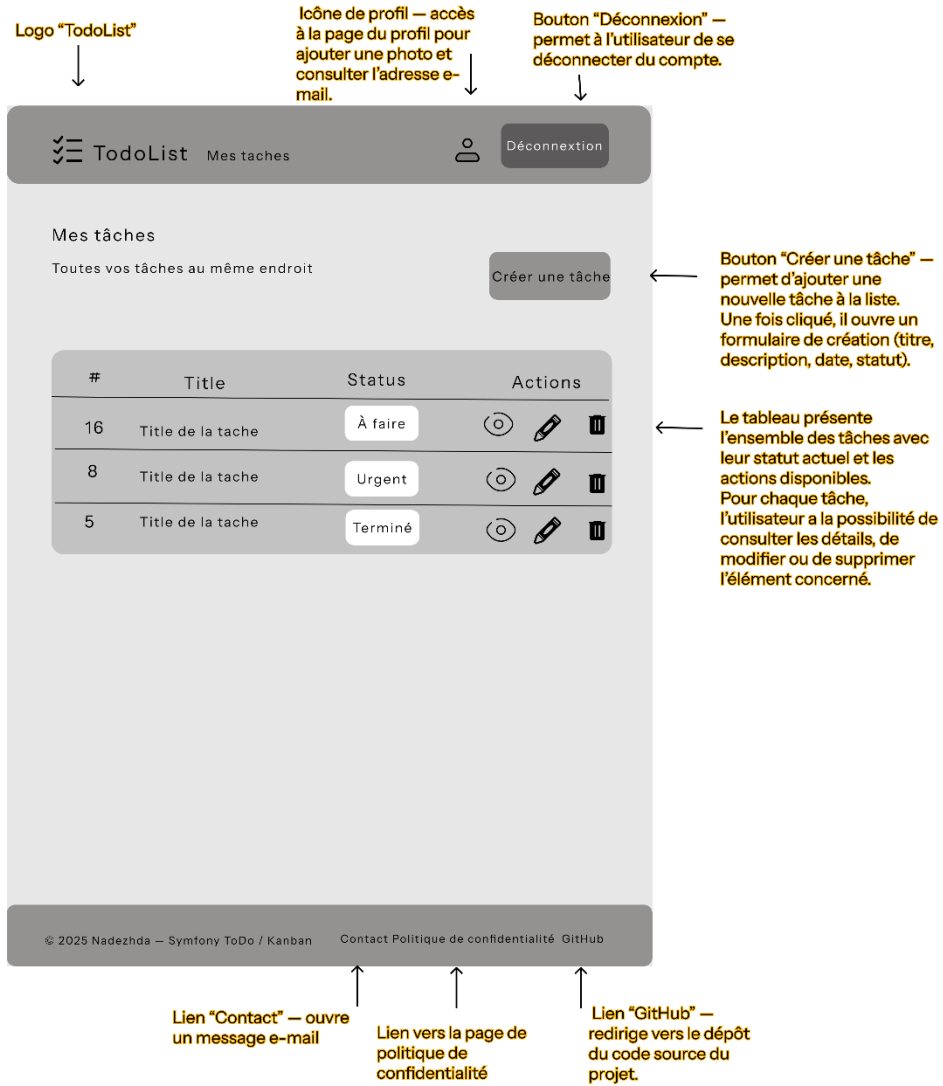
iPhone- la page kanban



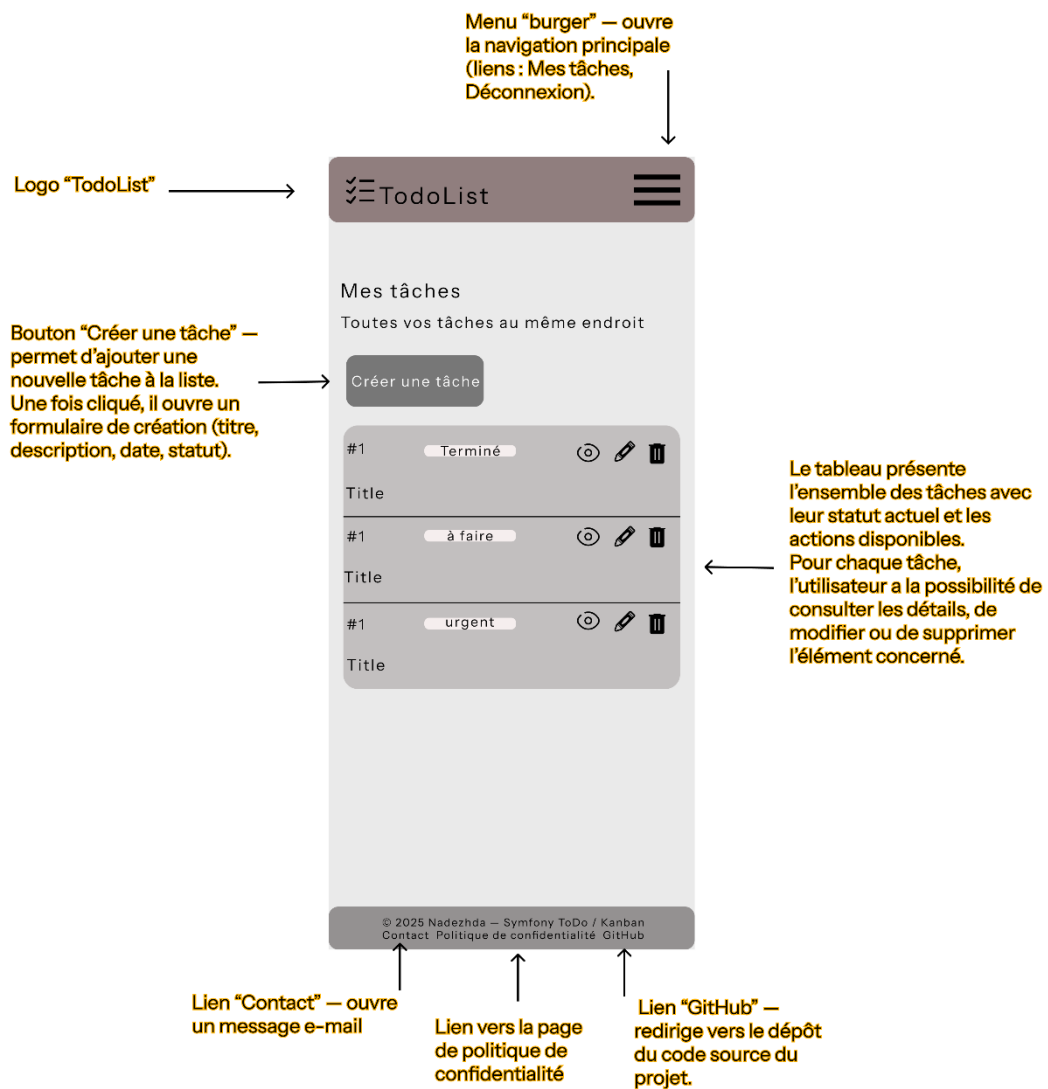
Desktop – la page tâches



Tablette-la page tâches



iPhone-la page tâches



Annexe B – Maquettes supplémentaires

